

The Influence of Algorithms on the Diversity of Music Recommendations

cs-24-it-7-07

7th Semester Project





Title:

The Influence of Algorithms on the
Diversity of Music Recommendations

Theme:

Recommender Systems

Project Period:

Autumn Semester 2024

Project Group:

CS-It7, Group 7
cs-24-it-7-07

Participant(s):

Eske Klintø Larsen
Manuel Delgado Albino
Mathias Holm Mariager
Matilde Schade Vaz Tocha
Thormod Jakob Schøn Merstrand
Toms Vanders

Supervisor:

Peter Dolog

Page Numbers: 44

Date of Completion:

December 19, 2024

Abstract:

Overfitting is a recurring problem within recommender systems that can lead to users missing out on relevant recommendations. This project aims to tackle the overfitting problem by analyzing the impact of different recommender models on playlist diversity in regards to the inner and outer diversity. Using content-based (Autoencoder) and collaborative filtering models (BPR, ALS, and NMF) along with a variety of evaluation metrics, we evaluated the trade-offs between quality and diversity. From our results we saw that the collaborative filtering models (BPR and ALS) exceeded the content-based models when it came to the evaluation metrics and creating diversity. We concluded that there are many different ways to interpret diversity for the user and to get a clear view of what model is the optimal one for diversity, we would have to perform future user studies aimed at discovering this. These findings contribute to improving algorithmic design, enhancing user satisfaction through personalized yet diverse recommendations.

Contents

1	Introduction	1
1.1	Approach	1
2	Background	3
2.1	Recommender System Algorithms	3
2.1.1	Content-Based Filtering	3
2.1.1.1	Neural networks	4
2.1.1.2	Autoencoder	6
2.1.2	Collaborative Filtering	7
2.1.2.1	Matrix Factorization	7
2.1.2.2	Non-Negative Matrix Factorization	8
2.1.2.3	Alternating Least Squares	10
2.1.2.4	Bayesian Personalized Ranking	11
2.2	Diversity	11
2.3	Song Recommender System	12
3	Approach	14
3.1	The datasets	14
3.2	Data preparation	16
3.3	UCloud virtual server	16
3.4	Algorithm Implementation	16
3.4.1	Environment	17
3.4.2	Content-based Filtering	17
3.4.2.1	Preprocessing of data	18

3.4.2.2	User profile	19
3.4.2.3	Basic Neural Network	20
3.4.2.4	Autoencoder	21
3.4.2.5	Base case	22
3.4.3	Collaborative Filtering	23
3.4.3.1	Bayesian Personalized Ranking	24
3.4.3.2	Non-negative Matrix Factorization	26
3.4.3.3	Alternating Least Squares	27
3.5	Evaluation	29
4	Results	33
4.1	Precision@K and Recall@K	34
4.2	Hit@K	35
4.3	NDCG@K	36
4.4	MAP	37
4.5	Diversity	38
5	Discussion	39
5.1	Usability	39
5.2	Scalability	40
5.3	Security	41
6	Future Work	42
7	Conclusion	44
	Bibliography	45
A	BPR Hyperparameter Experiment	a
B	ALS NonNegative Parameter Experiment	b
C	NMF Experiment with Binary Transformation and Negative Sampling	c
D	Random Recommender Implementation	d

Chapter 1

Introduction

Recommender systems have become a key player in what makes a good online service. Whether it be streaming services like Netflix or Spotify trying to recommend the next movie or song, to online shopping services like Amazon or Temu recommending you the next thing to buy. Common for all these services is their aim to give the best recommendations they can while fitting the users preferences as much as possible. However, a problem that has emerged from this is the overfitting problem [1]. This problem comes from the algorithmic nature of the recommender system models, always trying to find the best fit for the user based on the model's parameters. Eventually, the model may end up with the exact users preferences and therefore tailor recommendations specifically to these. In itself, this does not seem like that big of a problem, but having the model become overfitted may result in it missing out on potential recommendations for the user that lay outside what the model knows the user prefers. To mitigate this problem, researchers have started to implement diversity metrics in their recommender systems to make sure recommendations reach further out than what is only the perfect fit for the user. [1]

1.1 Approach

This project explores the diversity of recommendations made by different recommender system models. The models in question were selected from within both content-based filtering and collaborative filtering so that multiple aspects of recommender systems could be explored. For content-based models, we worked within the field of neural networks and chose to work with both an autoencoder model and one that was not autoencoded. The collaborative filtering models lay within the field of matrix factorization and were Non-negative matrix factorization (NMF), Alternating least squares (ALS), and Bayesian personalized ranking (BPR). Along with choosing the models to test, we decided that the evaluation metrics used in the testing should be: Precision@K, Recall@K, NDCG@K, MAP, Hit@K, and diversity, where the latter would be the main focus of the project. We also chose to not only look at the diversity scores but also include the works of Robinson et al. [2], who had found that diversity could be divided into inner and outer diversity.

To test the models we used two datasets that we obtained from the *Million Song Dataset + Spotify + Last.fm* Kaggle page [3]. The *User Listening History* dataset is used with the collaborative filtering algorithms. The content-based algorithms use both the *Music Info* dataset to find similar songs and the *User Listening History* dataset to get a user profile.

The datasets were prepared for testing by transforming them and splitting them into train and test sets. Due to the size of the datasets we had to use the services of UCloud [4] who provided us with the necessary computing power to run the models on the datasets. Each model was tested by having it create three lists of recommended songs for the user, with the sizes of the lists being 10, 25, and 50 songs. A model to recommend random songs (Appendix D) was also added in the testing phase to be able to compare its performance to the implemented models.

Our work with recommender models revealed that diversity encompasses many facets and different models employ various strategies to achieve diversity. We observed the collaborative filtering models produced more diverse recommendations compared to the content-based models but noted that there were still limitations for both types of models. Another key aspect of our research was choosing the right model for the dataset is important, we especially saw this for the NMF model struggling to work with implicit feedback. Furthermore, we acknowledged that due to how differently diversity can be interpreted we would have to do future research in the form of user testing to get a real insight into what model ultimately creates the best diversity.

Chapter 2

Background

2.1 Recommender System Algorithms

A recommender system is a system that aims to provide the user with information about items of interest within a given field through the use of algorithms. The system's effectiveness hinges on its algorithm's ability to correctly represent the user's current interests [5]. In pursuit of this, researchers have come up with a variety of algorithms to predict the user's interests. In the following sections, we will dive into some of the relevant ones within the content-based and collaborative filtering fields.

2.1.1 Content-Based Filtering

Content-based filtering is one of the most widely used recommender algorithms, and it is utilized by companies like Google [6]. This method generates recommendations by focusing on item-specific attributes within a dataset, such as genre, tempo, or BPM in a music context, to analyze item characteristics independently.

Advanced techniques transform item features into a latent feature space, enabling the usage of metrics to measure similarity. In this space, similarity measures like cosine similarity or Euclidean distance can effectively measure the relationships between items, leading to more accurate recommendations [7].

One major advantage of content-based filtering is that it mitigates the cold start problem, as it doesn't require extensive user data to make recommendations, just item features. This also improves the accuracy for sparse datasets where collaborative filtering (see chapter 2.1.2) would struggle due to a lack of user overlap. However, content-based filtering can be limited by solely relying on item features; it often lacks the diversity of recommendations that collaborative filtering can offer, as collaborative approaches can introduce new, unexpected items through patterns in user behavior.

2.1.1.1 Neural networks

Neural networks refer to a subset of models in the field of machine learning. They are inspired by the biological structure of the human brain to enable the computer to make decisions based on data. [8]

Neural networks are built around using multiple layers, each layer can be categorized into three types: Input layer, hidden layer, and output layer. The Input layer handles the initial data and sends it to the next layer. The hidden layers are the layers between the input and output layers. These layers are used to process and transform the input data to find meaningful patterns in the data. Finally, the output layer uses the data from the hidden layers to make a prediction output. [9]

Each layer consists of several neurons (nodes). Each neuron receives input data from the previous layer and applies a unique weight to each input. It then calculates the sum of all the inputs and adds a bias. It will then apply an activation function to this result to determine the output of that neuron. [9]

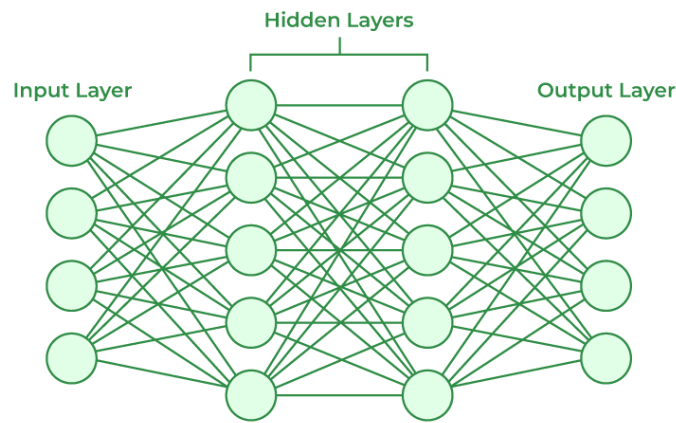


Figure 2.1: Illustration of a simple neural network [10]

The neurons apply an activation function before giving an output. Many activation functions are non-linear since many real-world patterns are not linear and only having the weighted sum transformation of the data would limit the ability to find patterns in complex data patterns [11]. There exist many different activation functions that can be applied and each layer of neurons can have different activation functions. This means the hidden layers may use one form of activation function to efficiently fit the data patterns and the output has another to predict the data based on what the goal is for the prediction.

For the hidden layers, a common activation functions used for deep learning is the Rectified Linear Activation (ReLU) activation function. The ReLU activation function is a simple function that adapts well to most data and is less likely to have problems surrounding vanishing gradients like other popular activation functions such as the sigmoid or the Tanh activation functions. The ReLU is defined as [11]:

$$\text{ReLU}(x) = \max(0, x)$$

This means that if given an input below 0, it will return 0, and for values above, it will simply return the value of the weighted sum plus the bias. This means that it will be able to compute more complex patterns that are not linear.

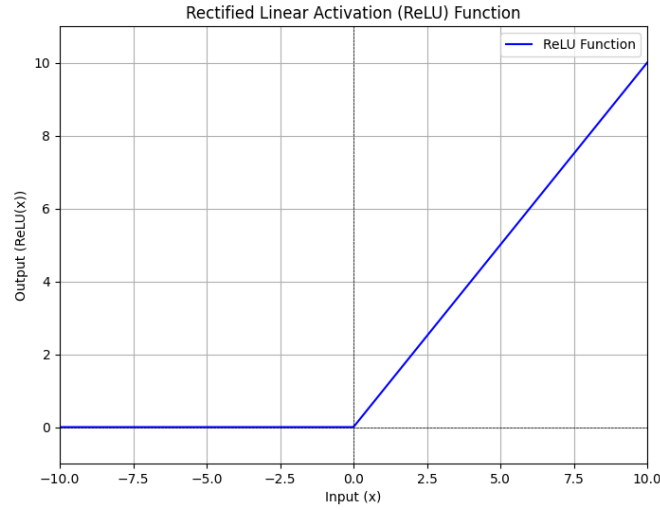


Figure 2.2: Plot of the ReLU activation function

The output layer often implements a different activation function than the hidden layers, since the output layer does not try to fit a pattern but rather tries to make a prediction based on the pattern computed in the hidden layers. Many different activation functions can be used on the output layer depending on the target of the prediction. For example in cases of multi-classification, the Softmax can be used [12], and in cases of binary classification the sigmoid function can be applied.

The Softmax activation function is defined as followed [9]:

$$\sigma_i = \frac{e^{z_i}}{\sum_{j=1}^{j=K} e^{z_j}}$$

In simple words, the Softmax activation function will take a vector of inputs and convert it to a probability distribution that all sum to one. This means that it will give a probability to each of the possible classes to predict between.

The Sigmoid activation function is defined as followed [9]:

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

The Sigmoid activation function will follow an s-formed curve and convert any given input to an output between zero and one. this makes it suitable for probability prediction between two classes (binary classification) using a threshold.

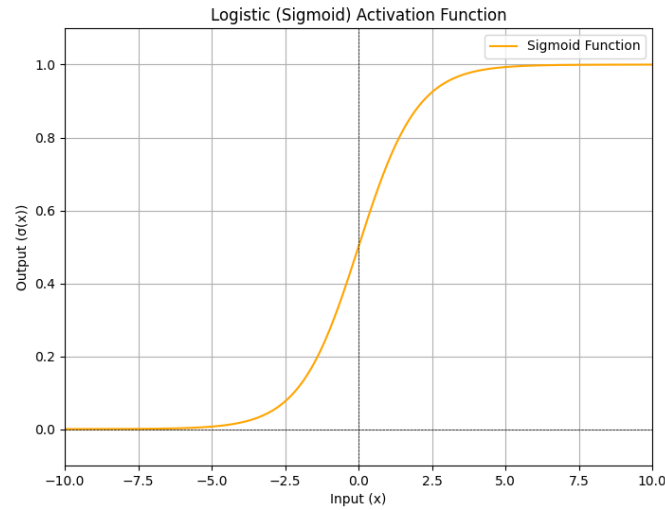


Figure 2.3: Plot of the Sigmoid activation function

2.1.1.2 Autoencoder

An autoencoder is a deep learning architecture designed to encode input data into a lower-dimensional representation (compression) and afterward reconstruct the compressed data to its original dimensions (decompression). The encoder identifies the patterns of the input data, while the decoder uses this compressed representation to recreate the original data. Autoencoders are applied to various types of applications, including image processing, recommender systems, data compression, generative modeling, and audio or speech processing. The figure below illustrates the autoencoder framework. [13]

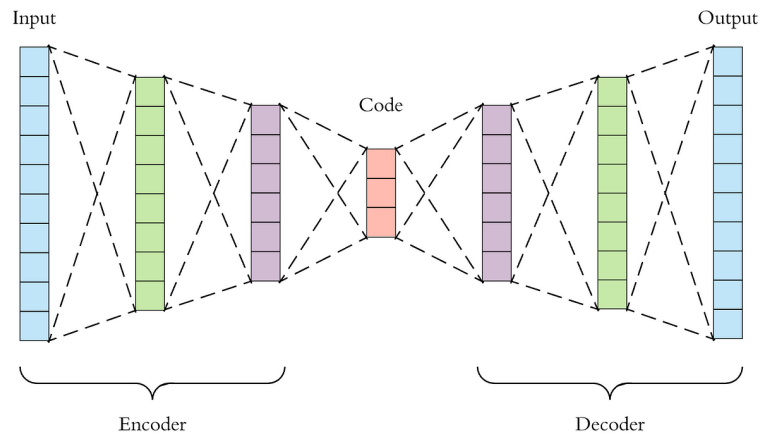


Figure 2.4: Illustration of an Autoencoder [14]

Autoencoders are known for denoising their input. In this process, the model is trained on a dataset where noise (random data meant to distort the model) is intentionally added to the input while the target output remains the original, noise-free version. During training, the encoder learns to focus on the important attributes of the data and disregard the noise. This enables the autoencoder to reconstruct clean data from noisy inputs. Such denoising capabilities are especially valuable in tasks such as image restoration and speech enhancement. By filtering out irrelevant variations introduced by noise, autoencoders

improve the quality of reconstructed data. [13]

2.1.2 Collaborative Filtering

Collaborative filtering is one of the earliest and simplest techniques used in Recommender Systems. Unlike content-based methods, collaborative filtering is based on feedback provided by other users rather than the attributes or features of the items themselves [15]. The fundamental idea is that if two users have rated items similarly in the past, their preferences for unseen items are also likely to align [16].

Ricci et al. [16] stated that this approach offers some significant advantages over content-based methods, for instance, it can recommend items even when detailed content information is unavailable or difficult to obtain, relying instead on user feedback. Furthermore, recommendations are based on peer evaluations of item quality rather than content, which may not always accurately reflect quality. Collaborative filtering also allows for diverse recommendations, suggesting items with vastly different content as long as they have received positive attention from others.

According to Nikolakopoulos et al. [17] collaborative filtering methods are divided into neighborhood-based and model-based approaches. They clarify that neighborhood-based methods directly use user-item ratings to predict a users rating for unseen items, while model-based methods train a predictive model on these ratings. The model captures key characteristics of users and items, enabling it to forecast ratings.

Model-based approaches are advantageous as they identify complex patterns, handle outliers better, and generalize more effectively, resulting in more accurate and reliable recommendations [17].

2.1.2.1 Matrix Factorization

Factorization methods are a model-based approach that addresses issues such as lack of data and limited coverage. They map users and items into a smaller feature space that highlights their key traits, allowing meaningful relationships to be identified, even if users have rated different items [15].

Nikolakopoulos et al. [17] explain that factorization techniques are commonly applied in two ways to improve recommender systems: (1) factorizing a sparse similarity matrix and (2) factoring a user-item rating matrix. Since the focus of this work is on the latter, we will limit our discussion to that approach.

Matrix factorization techniques address issues like cold-start and limited coverage by factorizing the user-item rating matrix R [17]. Nikolakopoulos et al. [17] clarify that the goal is to approximate R , of size $|U| \times |I|$, as

$$R \approx P \cdot Q$$

where $P \in \mathbb{R}^{|U| \times k}$ represents user factors, and $Q \in \mathbb{R}^{|I| \times k}$ represents item factors, with $k \ll \min(|U|, |I|)$. The matrices P and Q are found by minimizing the error:

$$E(P, Q) = \sum_{(u,i) \in R} (r_{ui} - p_u q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2)$$

The regularization term (λ) prevents overfitting. When ratings are sparse, only observed values of R are used in the optimization.

Singular Value Decomposition (SVD), a foundational method in linear algebra, is often associated with matrix factorization [18]. Assuming a system with m users and n items, SVD decomposes the score matrix $R \in \mathbb{R}^{m \times n}$ into three components:

$$R = U \cdot \Sigma \cdot V^T,$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal matrices containing the singular vectors of R , and $\Sigma \in \mathbb{R}^{k \times k}$ is a diagonal matrix containing the singular values of R in descending order, with k being the rank of R [19]. According to Koren et al. [18], this decomposition effectively reduces the rank of the matrix, capturing the most significant patterns in the data.

However, despite its theoretical appeal, the direct application of the algebraic SVD method to Collaborative Filtering is problematic due to the sparsity of user-item rating matrices [18]. First, SVD requires a complete matrix as input, which is rarely available in real-world datasets, and filling missing values through imputation introduces computational overhead and can distort the original data distribution [18, 19]. Second, directly applying SVD to the observed entries without proper regularization risks overfitting, as the model may overly conform to the limited available data, compromising its ability to generalize to unseen interactions [18].

To overcome these limitations, modern variations of SVD, such as Funk-SVD or SVD++, focus on factorizing only the observed entries of the matrix, and these methods incorporate regularization terms to enhance generalization and prediction accuracy [19]. These advancements have also enabled the integration of implicit feedback, such as user behavior and preferences, into the modeling process, supporting a wide range of applications and datasets [15].

In the next sections, we will explore some matrix factorization techniques that better suit our problem.

2.1.2.2 Non-Negative Matrix Factorization

Non-negative Matrix Factorization (NMF) is a popular matrix factorization technique for increasing the recommendation accuracy of collaborative filtering algorithms used in the development of recommender systems among other fields [20]. It decomposes a non-negative user-item matrix into lower-rank non-negative matrices and, like other matrix factorization models, tries to uncover hidden factors representing the relationship between user preferences and item characteristics. The non-negativity constraint leads to additive combinations of factors, making them easier to interpret since the meanings of negative values are often not straightforward [21]. Non-negativity also naturally aligns with the data available in recommender systems such as ratings, play counts, or purchase histories which are inherently non-negative. Together with better interpretability, this often makes NMF a more appropriate choice for recommender systems than other popular matrix factorization techniques like Singular Value Decomposition (SVD) [22].

The goal of NMF is to minimize the difference between the original matrix V and the product WH [23]. Given a non-negative matrix V of size $m \times n$, NMF seeks to approximate

V as the product of two lower-rank non-negative matrices W (size $m \times k$) and H (size $k \times n$), such that:

$$V \approx W \cdot H$$

There are different approaches to NMF, primarily differing in the objective functions used to measure the divergence, as well as in the optimization algorithms used to compute the factorization. Two common divergence measures for NMF are the Frobenius norm (squared error) and an extension of the KullbackLeibler (KL) divergence tailored for positive matrices [23].

The optimization problem using the Frobenius norm can be formally stated as:

$$\min_{W, H} \|V - WH\|_F^2 \quad \text{subject to} \quad W \geq 0, H \geq 0$$

For NMF, hidden factors are usually found through optimization techniques that utilize iterative update rules. Multiplicative update (MU) rules, introduced by Lee and Seung [23], have been a popular method due to its simplicity and ease of implementation. The MU algorithm updates W and H matrices element by element, ensuring non-negativity is maintained throughout the optimization process:

$$H_{ij} \leftarrow H_{ij} \cdot \frac{(W^T V)_{ij}}{(W^T W H)_{ij}}$$

$$W_{ij} \leftarrow W_{ij} \cdot \frac{(V H^T)_{ij}}{(W H H^T)_{ij}}$$

In addition to the MU approach, other widely used optimization algorithms for NMF include Stochastic Gradient Descent (SGD) and Alternating Least Squares (ALS). Gradient Descent is the most straightforward optimization method due to its simplicity. However, it often suffers from slow convergence, especially with large datasets. More advanced techniques like conjugate gradient methods can achieve faster convergence rates, but they are significantly more complex to implement. SGD addresses these issues [24]. On the other hand, ALS performs comparably to MU in terms of speed but can experience unstable convergence because it enforces non-negativity by simply setting negative values to zero during updates [20].

There is extensive research exploring NMF and its various adaptations to optimize recommendation outcomes. For instance, a study by Bobadilla et al. [21] compares several matrix factorization approaches, including Probabilistic Matrix Factorization (PMF), Biased Matrix Factorization (BiasedMF), and Bayesian NMF (BNMF). The findings indicate that while BNMF excels in prediction accuracy, standard NMF offers adequate results in precision and delivers the best performance in beyond-accuracy metrics, promoting diversity and novelty within recommendations. Furthermore, research by Wundervald [25], focused on fairness in music recommender systems, highlights that NMF effectively reduces biases by decreasing the correlation between item popularity and recommendation frequency. This indicates that NMF not only enhances recommendation accuracy but also contributes to mitigating inherent data biases and promoting diverse and novel recommendations.

Additionally, there is research that builds upon the standard NMF framework. For example, Haque et al. [24] propose Divergence-Based Non-negative Matrix Factorization (TNMF) by integrating Generalized Kullback-Leibler (KL) divergence as the objective function and employing SGD optimization algorithm to enhance scalability and efficiency when handling large and sparse datasets. In their results, TNMF outperforms other methods like Probabilistic Matrix Factorization (PMF), Bayesian Probabilistic Matrix Factorization (BPMF), and ALS-WR on half of the evaluated datasets.

Numerous studies demonstrate NMFs adaptability and effectiveness in balancing key metrics such as accuracy, diversity, novelty, and fairness, making it instrumental in advancing recommender systems.

2.1.2.3 Alternating Least Squares

Alternating Least Squares (ALS) is a widely-used matrix factorization technique for solving recommendation problems, particularly in the context of implicit feedback. ALS builds on the foundation of factorizing the user-item interaction matrix R into two lower-dimensional matrices P (user features) and Q (item features) to approximate $R \approx P \cdot Q^T$ [26, 27].

When explicit feedback is unavailable, implicit feedback signals such as clicks, views, or playcounts are used to infer user preferences. ALS handles implicit feedback by introducing a confidence parameter $c_{u,i}$, which reflects the reliability of an interaction. Observed interactions, such as a high playcount, are assigned higher confidence values, while unobserved interactions are assigned lower confidence. The optimization objective for ALS with implicit feedback is defined as:

$$\min \sum_{(u,i)} c_{u,i} \left(p_{u,i} - q_i^T p_u \right)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2),$$

where $c_{u,i} = 1 + \alpha r_{u,i}$ is the confidence level, $r_{u,i}$ is the implicit interaction score (e.g., playcount), and $p_{u,i}$ represents the binary preference indicator: $p_{u,i} = 1$ if $r_{u,i} > 0$, and $p_{u,i} = 0$ otherwise. The regularization term $\lambda (\|q_i\|^2 + \|p_u\|^2)$ mitigates overfitting by penalizing large values of the latent factors [26].

The iterative optimization process alternates between fixing Q while solving for P , and vice versa. This alternating approach simplifies the optimization problem, as fixing one variable transforms the problem into a convex least-squares problem, which is computationally efficient to solve [27].

A key advantage of ALS is its scalability. The iterative computation of Q and P can be parallelized or distributed, making it highly effective for large-scale datasets where the user-item interaction matrix is sparse. This scalability has made ALS a standard choice for recommendation tasks, especially in scenarios such as music recommendations, where implicit feedback is prevalent and the dataset size can be substantial [26].

2.1.2.4 Bayesian Personalized Ranking

Bayesian Personalized Ranking (BPR) is a state-of-the-art matrix factorization method that produces user and item vectors with significantly reduced dimensionality. It learns the most representative features of items and user profiles from implicit rating data. [28].

The core principle of BPR is to optimize for the ranking of items rather than their absolute scores. Given a set of observed interactions $S \subseteq U \times I$, where U represents the set of users and I represents the set of items, the goal of BPR is to derive a personalized ranking $>_u$ for each user u . BPR assumes that if a user u has interacted with an item i , the user is more likely to prefer i over another item j that they have not interacted with. This preference is represented as $i >_u j$ [28].

To achieve this, BPR defines an optimization criterion, BPR-Opt, derived from a Bayesian analysis of the problem. The likelihood function $p(i >_u j \mid \Theta)$, where Θ is the parameter vector of the model, represents the probability that user u prefers item i over item j . This likelihood is modeled using a logistic sigmoid function:

$$p(i >_u j \mid \Theta) = \sigma(x_{ui} - x_{uj}),$$

where x_{ui} and x_{uj} are the predicted scores for items i and j , respectively. The sigmoid function ensures that the likelihood remains in the range $[0, 1]$, reflecting the degree of preference [28].

The optimization problem is formulated as maximizing the posterior probability $p(\Theta \mid >_u)$, which combines the likelihood of the observed rankings with a prior distribution on the model parameters. The resulting objective function, BPR-Opt, is expressed as:

$$\text{BPR-Opt} = \sum_{(u,i,j) \in S} \ln \sigma(x_{ui} - x_{uj}) - \lambda \|\Theta\|^2,$$

where the first term encourages correct pairwise rankings, and the second term is a regularization term to prevent overfitting [28].

To optimize BPR-Opt, a stochastic gradient descent algorithm, LearnBPR, is employed. LearnBPR iteratively samples triplets (u, i, j) from the observed interactions and updates the model parameters using gradient descent. [28].

The BPR framework has been successfully applied to various collaborative filtering models, including matrix factorization, enhancing their ability to generate personalized rankings [29]. Its emphasis on pairwise preferences overcomes the challenges posed by implicit feedback, providing a robust foundation for building recommender systems.

2.2 Diversity

Diversity plays a vital role in recommender systems, as a means to go beyond a singular focus on accuracy to encompass broader user satisfaction. It is a topic long studied by researchers to combat the over-fitting problem [1], which can lead to *echo chambers* or *filter bubbles* [30] where users are exposed only to familiar or narrowly tailored content [2]. This risk of over-personalization has drawn attention to the importance of diverse recommendations as a safeguard against these limitations, allowing users to explore a wider range of content that keeps them engaged and continuously discovering new interests.

Within the field of diversity for music recommendation specifically, Robinson et al. [2] have proposed the distinction between inner and outer diversity. They found that users had a clear distinction between diversity within their existing preferences, this being labeled as inner diversity, and outer diversity, which involved being introduced to music outside the user's usual listening habits. Robinson et al. [2] also found that the level of diversity the users wanted varied greatly as well as whether they preferred inner or outer diversity and more work would have to be done to generalize their findings.

2.3 Song Recommender System

Since this project aims to explore diversity in recommender systems, we chose to create song recommendations since it was a field where creating diversity in the recommendations could be relevant to helping users discover new music. It was also a field where we would have the option to explore both content-based and collaborative filtering algorithms and their impact on diversity. Having chosen the field of song recommendation we found it relevant to look at how the current actors in the field do these with one of these actors being the music streaming service Spotify [31].

Spotify considers a variety of metrics when they create song recommendations for a user. One of their more content-based approaches to recommending songs is creating a *taste profile* for the user [32]. The *taste profile* is based on the user's engagement with songs through searching, listening, skipping, or saving them. This way Spotify learns what songs the user likes to engage with and can this way recommend similar content that the user may end up liking. Other than the *taste profile* Spotify also touches on collaborative filtering algorithms since they take trends and similar users into consideration when making recommendations [32].

The recommendations created by Spotify are presented to the user in a variety of ways with one such way being the *Discover Weekly* playlist, where Spotify every week creates a 30-song playlist of recommended songs for the user to listen to. Another way recommendations are presented is for the user's playlists, where Spotify gives the user the option of some songs that were found to be similar to that of the current content of the playlist [31].

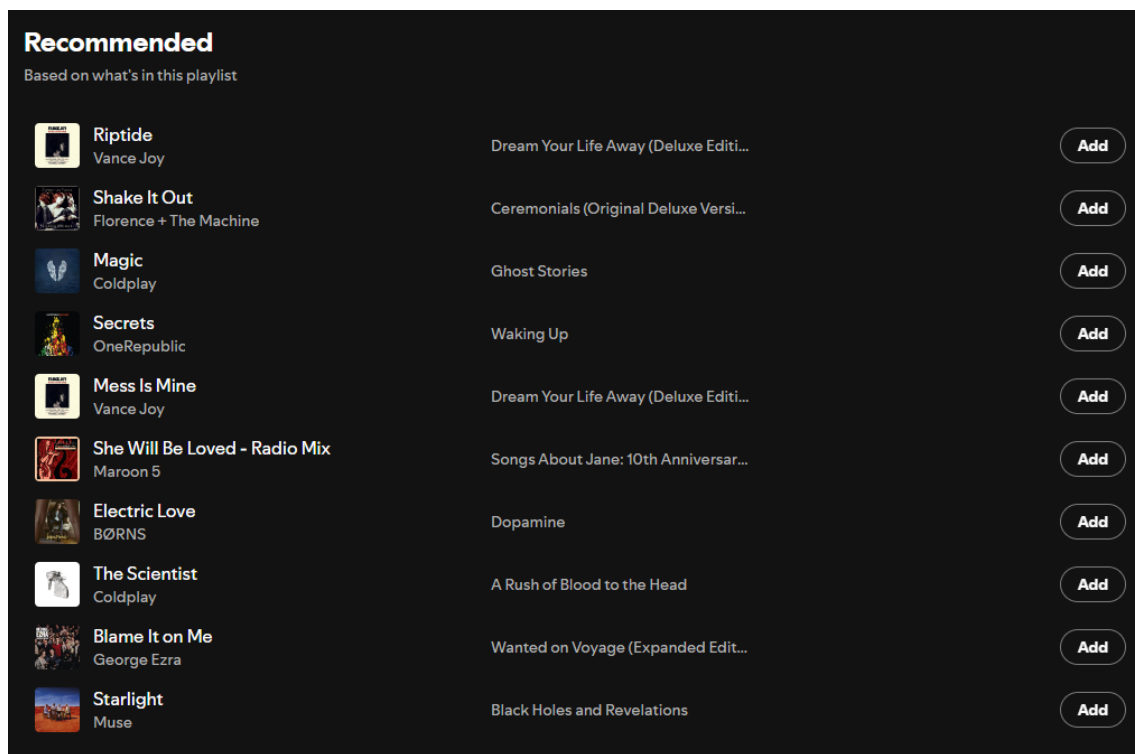


Figure 2.5: Spotify recommending songs for the user's playlist [31].

Chapter 3

Approach

In this chapter, we will explain our implementation. Within the following sections, the datasets and their different parameters are analyzed, providing us with an introduction to data preparation. After this, we delve into how we prepared each dataset for the implementation of the different algorithms. Finally, we discuss the metrics applied to conduct our research on diversity.

3.1 The datasets

The datasets we used for testing the content-based and collaborative filtering algorithms were provided by the *Million Song Dataset + Spotify + Last.fm* Kaggle page [3]. From this page, we used both the *User Listening History* dataset and the *Music Info* dataset for the content-based algorithms. For the collaborative filtering algorithms, we only used the *User Listening History* dataset.

The *User Listening History* dataset contained information about how often a user played a specific song. Each entry in this dataset had the following variables:

Variable name	Description
track_id	A unique ID given to the track within the dataset.
user_id	A unique ID given to the user within the dataset.
playcount	The amount of times the user had played the specific song.

Table 3.1: Description of Variables in the *User Listening History* dataset [3]

The *Music Info* dataset contained information about a variety of music tracks along with their corresponding metadata. Each entry in this dataset had the following variables:

Variable name	Description
track_id	A unique ID given to the track within the dataset.
name	The name of the given song.
artist	The name of the artist behind the given song.
spotify_preview_url	An URL containing a 30-second audio preview of the song.
spotify_id	The unique ID of the song on Spotify.
tags	A list of tags to describe the song. This could be things like: <i>rock</i> , <i>alternative</i> , <i>pop</i> or <i>90s</i> .
genre	The genre that best describes the song.
year	The release year of the song.
duration_ms	Duration of the song in milliseconds.
danceability	A floating-point number between 0 and 1 that describes how suitable a song is for dancing, with 1.0 being the most danceable.
energy	A floating-point number between 0 and 1 that describes how energetic the song is. Tracks that feel fast, loud, and noisy will score closer to 1.
key	An integer mapped to a pitch using standard Pitch Class notation. A value of 0 will for example correspond to C.
loudness	A floating-point number ranging between -60 and 0 db that describes the overall loudness of the song. The more negative the value is the more quieter and softer sounding the song is.
mode	An integer indicating the modality of the song, with 1 being major and 0 being minor.
speechiness	A floating-point number between 0 and 1 that describes the presence of spoken words in a song, with a higher number meaning the song is closer to being exclusively speech-like.
acousticness	A confidence measure between 0 and 1 describing whether the song is acoustic. A value of 1 represents high confidence that the song is acoustic.
instrumentalness	A floating-point number between 0 and 1 representing whether the song contains no vocals. The closer the value is to 1 the greater the chance the song contains no vocals.
liveness	A floating-point number between 0 and 1 representing the presence of an audience in the song. Higher values mean there is a greater chance of the song being played live.
valence	A floating-point number between 0 and 1 describing the musical positiveness of the song. Higher values mean the song is more positive sounding in the form of sounding happy, cheerful, or euphoric.
tempo	The overall estimated beats per minute of the song.
time_signature	The estimated time signature of the song.

Table 3.2: Description of Variables in the *Music Info* Dataset [3] [33]

3.2 Data preparation

Before the testing of our recommender system algorithms began, we prepared our data in various ways. Since the dataset contained strings that were not numerical we had to remap these into numerical values to be compatible with our models.

Furthermore, we also removed some variables from the *Music Info* dataset during this process. These were the *spotify_preview_url* and *spotify_id* variables since these would not be relevant to use with our algorithms because these would only be used in a Spotify context.

Another part of the preparation was splitting our data into training and test data, which is also a common practice within the machine learning space to avoid overfitting. This made sure our models would perform better in real-world scenarios with unseen data [34]. We chose to go with an 80/20 train/test split for the two datasets where we used random splitting for the *Music Info* dataset. The *User Listening History* dataset had more things to take into consideration when splitting it. This dataset contained implicit feedback in the form of *playcount* meaning that, whereas a dataset with explicit feedback, which could be a rating from one to five, would be able to indicate a user's preference, we instead had *playcount* which only indicates the confidence of a user liking something [35]. Keeping in mind the noise that could exist from having implicit feedback [35], we chose to remove all users who had less than 50 interactions within the dataset to make sure we had enough data on the users that were left in. Furthermore, we used stratified splitting on the remaining data to make sure all user IDs would be represented in both the train and test datasets.

3.3 UCloud virtual server

Having prepared our data, we moved on to try and test our algorithms with the datasets but found that we did not have the necessary hardware to run the tests effectively. For instance, even after having removed multiple rows from the *User Listening History* dataset, the combined rows of the test and train datasets were 1.8 million, and with a maximum of 32 GB RAM at our disposal, this would not cover our needs for testing on the dataset. Therefore, we got access to UCloud [4] which is an online service that provides high computing power tools to its users. It was created by the University of Southern Denmark (SDU), and its goal is to support the needs researchers may have when it comes to computing and data management through a user-friendly and intuitive interface [36]. Through UCloud we got access to the AAU/K8 CPU and GPU nodes [37] that provided us with all the computing power we would need.

3.4 Algorithm Implementation

By now we have outlined both the datasets and the transformations applied and will now talk in depth about our implementation details, ranging from used libraries and models to leveraged parameters and eventually the implementation of the algorithms.

3.4.1 Environment

While programming the algorithms, we used Python’s version 3.9.20 on both our personal computers and the UCloud server, making it easier for us to minimize the dependency errors between different package versions.

Since UCloud’s framework does not allow the terminal environment to persist, every time a new job is created and a terminal is opened on the virtual server to run code, all necessary tools must be reinstalled, with Ubuntu Noble and Python 3.12.7 as the base version. To prevent the time-consuming task of not only uninstalling Python to install the correct version but also manually installing every pip package and Java dependency, we built a script (Figure 3.1) that installs Miniconda [38], a lightweight, free version of the Anaconda Distribution [39] that includes only the essential components: Conda, Python, their dependencies, and a minimal set of additional useful packages. With this, we are able to create an environment with our project’s Python version and every requirement necessary for the test to run smoothly.

```
1 mkdir -p ~/miniconda3
2 wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.
   sh -O ~/miniconda3/miniconda.sh
3 bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
4 rm -rf ~/miniconda3/miniconda.sh
5 ~/miniconda3/bin/conda init bash
6
7 # Wait 5 seconds before proceeding
8 sleep 5
9
10 # Relaunch Bash, ensuring it sources ~/.bashrc, then activate the Conda
   environment
11 exec bash --rcfile <(echo "source ~/.bashrc; conda create -n recsys
   python=3.9 -y; conda activate recsys; cd 1SemRecommenderSystem/; pip
   install -r requirements.txt; sudo add-apt-repository ppa:webupd8team/
   java; sudo apt-get install openjdk-8-jdk-headless -qq"
```

Figure 3.1: Setup script for Miniconda environment and package installation on UCloud’s terminal.

3.4.2 Content-based Filtering

In this section, we will go in-depth with our approach regarding the implementation of Content-based recommenders. These recommenders will be based on the feature data of the songs to recommend new songs to users utilizing the concept of item-to-item recommendation.

The content-based filtering process can be broken down into the following five key steps:

- **Preprocessing the item matrix:** Preparing the data for model training by cleaning and transforming the song features.
- **Defining user profiles:** Constructing individual user profiles based on their listening history and play count.
- **Model building:** Developing the recommender model using the user profiles and

song features.

- **Evaluating the models:** Assessing the performance of the models through appropriate metrics and validation techniques.
- **Selection and parameter tuning:** Selecting the best-performing model and fine-tuning its parameters to optimize recommendation quality.

3.4.2.1 Preprocessing of data

To prepare the track features dataset, *Music info* [3], the columns *genre* and *tags* had to be preprocessed to be compatible with the models. The *genre* is of categorical value while the tags are multi-valued containing a list of tags. To bypass this problem binary encoding is implemented for the values in these columns. For *genre* one-hot encoding is applied using the function *getdummies* from the pandas library [40]. For the *tags* the binary encoding was utilized with the MultiLabelBinarizer from the Sklearn library [41]. Assuming the dataset contains all possible genres and tags, with 15 unique genres and 99 unique tags, this preprocessing would add a total of 114 new columns to each track.

To further optimize the data for the model a scaler was fitted and transformed the data. specifically, the MinMaxScaler from the Sklearn library was implemented. This scaler will take the data values and transform them between a min and max value. In this case, the values were transformed in the default range from zero to one.

```

1 ...
2 import pandas as pd
3 from sklearn.preprocessing import MinMaxScaler, MultiLabelBinarizer
4
5 ....
6
7 def preprocess_data(data, scaler=MinMaxScaler()):
8     meta = data[["name", "artist", "track_id"]]
9     features = data.drop(columns=["name", "artist", "track_id"], errors="
ignore")
10    features["genre"] = features["genre"].astype("category")
11
12    # Handle tags, ensuring that we don't attempt to split non-string
values
13    features["tags"] = features["tags"].apply(
14        lambda x: (
15            eval(x)
16            if isinstance(x, str) and x.startswith("[")
17            else ([tag.strip() for tag in x.split(",")] if isinstance(x,
str) else []))
18        )
19    )
20
21    # Initialize MultiLabelBinarizer and fit it to the tags data
22    mlb = MultiLabelBinarizer()
23    tags_encoded = pd.DataFrame(
24        mlb.fit_transform(features["tags"]), columns=mlb.classes_
25    ).add_prefix("tag_")
26
27    genre_encoded = pd.get_dummies(features["genre"], prefix="genre")
28
29    # Combine the processed data
30    processed_data = pd.concat([features, genre_encoded, tags_encoded],
axis=1)
31    processed_data = processed_data.drop(
32        columns=["genre", "tags", "year"], errors="ignore"
33    )
34
35    # Scale the features
36    processed_data = pd.DataFrame(
37        scaler.fit_transform(processed_data), columns=processed_data.
columns
38    )
39
40    return processed_data, meta
41 .....
```

Figure 3.2: Preprocessing of data.

3.4.2.2 User profile

To recommend songs, we require some input from the user. This was addressed by creating a class that processes the rating matrix, which contains *user_id*, *track_id*, and *playcount*. Using the *user_id*, we retrieved a list of songs the user had listened to.

Next, we aggregated all the songs in the list to calculate an average value for each numeric attribute in the preprocessed dataset. Additionally, we implemented a second approach to

assess whether weighing the songs by *playcount* would enhance the results. This involved aggregating the songs as before but with weights applied based on the *playcount*.

```
1 import numpy as np
2
3 .....
4 def aggregate_user_preference(user_id, ratings, tracks):
5     user_ratings = ratings[ratings["user_id"] == user_id]
6     user_track_ids = user_ratings["track_id"].values
7
8     user_tracks = tracks.iloc[user_track_ids]
9
10    return np.mean(user_tracks, axis=0)
11 .....
```

Figure 3.3: User profile.

3.4.2.3 Basic Neural Network

To get a greater understanding of neural networks we implemented a basic neural network using the TensorFlow library [42]. TensorFlow is an open-source platform developed by Google to deploy ML models. TensorFlow allows for an easy introduction to new users but with the ability to expand to complex models. Since we did not have much experience with neural networks we decided to implement a basic one to understand the different aspects.

The idea behind the model was to use the softmax activation function (see section 2.1.1.1) to treat the recommendation problem as a classification problem where each available song for the user is a unique class. The neural network would then be trained on the feature data to find the probability of it being a specific song. For the recommendation, it would receive the aggregated user preference and give a list of probabilities for each song in the dataset. We found this to not be the optimal solution because of a few reasons. One reason is that treating each song as a unique class caused problems with the generalization of the model and creating a very static model that only works well if you have a fixed set of song data which in most real-world scenarios will not be realistic. This can also be seen in the training where the model has a high accuracy on the trained data but on the validation data will have very low to no accuracy indicating strong over-fitting of the model. This makes sense in the case that it can only be good at predicting the songs it has been trained on since it does not know the unique classes of songs outside that. This model approach would probably be better suited for classifying the different songs into for example genres, based on the song features. However, since the data was already classified and we wanted to focus more on recommending a list of songs we explored different approaches.

A simple approach to use a neural network for recommendation could be to have two classes: played and not played displayed as vectors for each user. This could be implemented by a model that would use data about: each song's features given as a vector, each user's preferred song features represented by a vector, and finally data about each user's interaction with each song. We tried to implement this as a simple model however the large amount of data caused a high memory usage. A solution for this could be to implement data batching in the training of the model and optimize data aggression. However, because of the time limitation of the project, we prioritized focusing on the autoencoder

model.

3.4.2.4 Autoencoder

The autoencoder (see section 2.1.1.2) was employed to reconstruct the empty fields in the dataset. Initially, it was trained using 80 percent of the tracks selected from the preprocessed data, which were passed into the encoder layer. In this layer, the data was compressed into 90 neurons, and optimized based on reconstruction loss. The compressed representation (90 neurons) was then passed into the decoder layer, which contained neurons equivalent to the original size of the preprocessed data (131 columns).

Subsequently, the number of training iterations (epochs) and batching were fine-tuned to optimize performance and mitigate over-fitting. Once the model was trained, the full preprocessed dataset was input into the autoencoder to estimate the missing values for each track.

With the reconstructed dataset containing no empty fields, we applied the cosine similarity formula provided by Scikit-learn [41] to find the nearest songs to an aggregated user input.

```
1 import numpy as np
2 import tensorflow as tf
3 from tensorflow.keras import Model, layers
4
5
6 class Autoencoder(Model):
7     def __init__(self, latent_dim=None, data=None, path=None):
8         super().__init__()
9         self.data, self.latent_dim = data, latent_dim
10        self.shape = data.shape[1:] if data is not None else None
11        if path is not None:
12            self.load(path)
13        elif self.shape:
14            self._build_models()
15
16        def _build_models(self):
17            self.encoder = tf.keras.Sequential(
18                [layers.Flatten(), layers.Dense(self.latent_dim, "relu")]
19            )
20            self.decoder = tf.keras.Sequential(
21                [layers.Dense(np.prod(self.shape), "sigmoid"), layers.Reshape
22                (self.shape)]
23            )
24            self.autoencoder = tf.keras.Sequential([self.encoder, self.
25            decoder])
26
27        def train(self, epochs=30, batch_size=512):
28            if not self.data:
29                raise ValueError("Data required.")
30            if not self.autoencoder:
31                self._build_models()
32            self.autoencoder.compile("adam", "mean_squared_error")
33            self.autoencoder.fit(self.data, self.data, epochs=epochs,
34                                batch_size=batch_size)
35        .....
```

Figure 3.4: Autencoder class.

image link: <https://towardsdatascience.com/applied-deep-learning-part-3-autoencoders-1c083af4d798>

3.4.2.5 Base case

We utilized the cosine similarity formula provided by Scikit-learn [41] to recommend based on non-encoded data to give a base case for testing the autoencoder. This worked using the aggregated user profile to compare the songs in the music dataset using the cosine similarity. The system would then recommend the top-k most similar songs to the aggregated user profile. This would allow us to compare the difference in recommending based on the reconstructed data of the songs from the autoencoder with the original format of the data.

```

1 import numpy as np
2 import pandas as pd
3 from sklearn.metrics.pairwise import cosine_similarity
4
5
6 class CosineRecommender:
7     def __init__(
8         self,
9         data: pd.DataFrame = None,
10        meta_data: pd.DataFrame = None,
11        user_data: pd.DataFrame = None,
12    ):
13        self.data = data
14        self.meta_data = meta_data
15        self.user_data = user_data
16        self.tracks_data_with_metadata = pd.concat(
17            [self.meta_data.reset_index(drop=True), self.data], axis=1
18        )
19
20    def __sort_by_distance(
21        self,
22        input_feature,
23        tracks_data,
24    ):
25        sim_scores = cosine_similarity([input_feature], tracks_data)[0]
26        sorted_indices = np.argsort(sim_scores)[::-1]
27        return sorted_indices
28
29    def recommend_similar_items(self, input_feature, user_id, top_n=None)
30    :
31        indices = self.__sort_by_distance(input_feature, self.data)
32        if user_id:
33            user_track = self.user_data[self.user_data["user_id"] ==
34            user_id][
35                "track_id"
36            ].values
37            length = top_n + len(user_track)
38            reduced_indices = indices[:length]
39            filtered_indices = [
40                item for item in reduced_indices if item not in
41                user_track
42            ]
43            if top_n:
44                final_indices = filtered_indices[:top_n]
45            return final_indices, self.tracks_data_with_metadata.iloc[
46                final_indices]

```

Figure 3.5: Cosine recommender class.

3.4.3 Collaborative Filtering

When discussing Collaborative Filtering we focused on Matrix Factorization algorithms, particularly Non-Negative Matrix Factorization, Bayesian Personalized Ranking, and Alternating Least Squares. To implement and test these we used the Microsoft Recommenders [43] repository on GitHub. With over 19000 stars and 3000 forks, this project's objective is to assist researchers, developers, and enthusiasts in prototyping, experimenting with, and bringing to production a range of classic and state-of-the-art recommender

systems.

The project offers a variety of knowledge due to being divided into five categories:

- **Preparation of data:** Preparing and loading data for each recommendation algorithm.
- **Model building:** Building models using various classical and deep learning recommendation algorithms.
- **Evaluation of models:** Evaluating algorithms with offline metrics
- **Model Selection and Optimization:** Tuning and optimizing hyperparameters for recommender models.
- **Operationalize models:** Operationalizing models in a production environment on Azure.

Furthermore, the program offers practical courses, starting with beginner-level *Quick-Start* sessions and progressing to more advanced *Deep-dive* content. This structure not only facilitated the initiation of our project but also offered us the essential resources to progress at the pace required for successful project completion with pertinent information.

With that being said, we employed their pip package *recommenders* and used their Cornac [44] and PySpark [45] implementations, two libraries that we will go through in the following chapters, to build the algorithms, taking into consideration the repository’s walk-throughs.

3.4.3.1 Bayesian Personalized Ranking

Cornac [44] was used to implement BPR. Cornac is an open-source Python framework dedicated to benchmarking and experimenting with a wide range of recommender algorithms. It supports both classical collaborative filtering techniques and more advanced methods, including models that leverage auxiliary information such as item descriptions or user-generated content [44]. The framework’s flexible API allowed for straightforward configuration and training of models, and reduced the overhead typically associated with setting up complex algorithms. It also seamlessly integrated with the Recommenders framework, for computing predictions and evaluation or ranking and diversity metrics. Furthermore, like Recommenders, Cornac is one of the frameworks endorsed by ACM RecSys 2023 for enhancing evaluation standards and ensuring reproducibility [44].

First, the data was prepared as described in Section 3.2, then the columns were reordered in order to fit Cornac’s expected format: user, item, rating (Figure 3.6).

```

1 # Read from file
2 test_listening_history = pd.read_csv(header=0, delimiter="\t",
    filepath_or_buffer="../remappings/data/dataset/
    test_listening_history_OverEqual_50_Interactions.txt")
3 train_listening_history = pd.read_csv(header=0, delimiter="\t",
    filepath_or_buffer="../remappings/data/dataset/
    train_listening_history_OverEqual_50_Interactions.txt")
4
5 # Reorder columns to (User, Item, Rating) - expected order for Cornac
6 train = train_listening_history[[COL_USER, COL_TRACK, COL_COUNT]]
7 test = test_listening_history[[COL_USER, COL_TRACK, COL_COUNT]]

```

Figure 3.6: Column change in data for BPR.

After, the BPR model was built, we then trained with *fit* and predicted with *predict_ranking*, a function available from the Recommenders [46] repository on GitHub that “Computes predictions of recommender model from Cornac on all users and items in data”. It is specifically “used for computing ranking metrics” (Figure 3.7).

To build the BPR model, we observed that no academic studies have determined the optimal hyperparameters for this model when applied to our database. Due to time constraints and limited computational resources, it is unfeasible for us to experiment with 192 diverse hyperparameter combinations, unlike the approach taken by Long et al. with ALS [47]. Consequently, we opted to adhere to the Recommenders [46] repository’s practical exercise and implement their suggested values. Although this exercise used a different database, we compared these hyperparameters to those defaulted by Cornac for BPR [48] and achieved superior results with the previously mentioned configuration, as shown in Appendix A, Table A.1.

```

1 NUM_FACTORS = 200
2 NUM_EPOCHS = 100
3
4 bpr = cornac.models.BPR(
5     k=NUM_FACTORS,
6     max_iter=NUM_EPOCHS,
7     learning_rate=0.01,
8     lambda_reg=0.001,
9     verbose=True,
10    seed=SEED)
11
12 # Training
13 with Timer() as t:
14     bpr.fit(train_set)
15 print("Took {} seconds for training.".format(t))
16
17 # Predicting
18 with Timer() as t:
19     all_predictions_bpr = predict_ranking(bpr, train, usercol=COL_USER,
20     itemcol=COL_TRACK, remove_seen=True)
21 print("Took {} seconds for prediction.".format(t))

```

Figure 3.7: Training and Prediction in BPR.

3.4.3.2 Non-negative Matrix Factorization

Just as with the BPR model, for NMF, we used the Cornac framework. Train and test data was reordered in (user, item, rating) format as seen in Figure 3.6. We adopted the same hyperparameter settings for NMF (Figure 3.8) as we did for BPR because, again, we did not have enough time for extensive hyperparameter optimization. While these settings improved over Cornacs default parameters [49], initial results were still poor. This was largely because Cornacs implementation of NMF is tailored for explicit feedback, whereas our user-song interactions measured by *playcount* are inherently implicit.

```
1 NUM_FACTORS = 200
2 NUM_EPOCHS = 100
3
4 nmf = cornac.models.NMF(
5     k=NUM_FACTORS,
6     max_iter=NUM_EPOCHS,
7     learning_rate=0.01,
8     lambda_reg=0.001,
9     verbose=True,
10    seed=SEED)
```

Figure 3.8: NMF Hyperparameters.

To address this issue, we explored ways to adapt NMF by transforming the data and incorporating negative sampling (Figure 3.9). First, before training we normalized all playcounts to a binary format (setting all observed user-song interactions to 1), effectively converting the implicit feedback into a simplified, implicit binary form. Next, we introduced negative sampling. For each user, we randomly selected a number of tracks they had not interacted with and added them to the training set with a value of 0. By binarizing the interaction data and adding negative samples we essentially can create an explicit feedback scenario. Using this version, the model can interpret interactions as positive signals (likes) and non-interactions as negative signals (dislikes), closely mirroring binary explicit ratings. Adding negative samples also reduced data sparsity. So it makes sense that these changes improved NMF performance, especially as we increased the number of negative samples per user (Appendix C).

```

1 # Transform playcount to binary
2 train[COL_COUNT] = 1
3
4 # Generate negative samples
5 def add_negative_samples(train, n_negative_samples):
6     users = train[COL_USER].unique()
7     items = train[COL_TRACK].unique()
8     existing_interactions = set(zip(train[COL_USER], train[COL_TRACK]))
9
10    negative_samples = []
11    for user in users:
12        user_items = set(train[train[COL_USER] == user][COL_TRACK])
13        potential_negatives = list(set(items) - user_items)
14
15        # Sample N negative items for the user
16        if len(potential_negatives) > n_negative_samples:
17            sampled_negatives = sample(potential_negatives,
18                                     n_negative_samples)
19        else:
20            sampled_negatives = potential_negatives
21
22        for item in sampled_negatives:
23            if (user, item) not in existing_interactions:
24                negative_samples.append([user, item, 0])
25
26    negative_df = pd.DataFrame(negative_samples, columns=[COL_USER,
27                                                         COL_TRACK, COL_COUNT])
28    return pd.concat([train, negative_df], ignore_index=True)
29
30 # Add negative samples to the training data
31 N_NEGATIVE_SAMPLES = 5
32 train = add_negative_samples(train, N_NEGATIVE_SAMPLES)

```

Figure 3.9: Adding Negative Samples to Dataset

Even though these transformations did provide some improvements, they also changed the dataset structure, making it difficult to compare NMF performance directly to other collaborative filtering algorithms under the exact same experimental conditions. We ultimately opted to run NMF in its base form using the original dataset. Although this choice resulted in NMF performance being notably worse than other models, it provided a fair benchmark and highlighted the importance of choosing algorithms aligned with the nature of your dataset.

3.4.3.3 Alternating Least Squares

The ALS algorithm was implemented using PySpark [45], the Python API for Apache Spark. Specifically, ALS was developed utilizing Spark’s Machine Learning Library (MLlib) [50], a scalable machine learning library built on top of Spark that offers a uniform set of high-level APIs. MLlib was selected because it supports model-based collaborative filtering, where users and items are represented by a small set of latent factors that can be used to predict missing rows. The `spark.ml` module uses the ALS algorithm to learn these latent factors. Furthermore, it accommodates implicit feedback by interpreting the input data as numerical values reflecting the strength of observed user actions (for example, the playcounts in our case), which are treated as levels of confidence in user preferences rather

than explicit ratings. The model subsequently identifies latent factors that predict a users expected preference for an item.

Before applying the ALS model, the data was preprocessed to conform to the requirements of the spark.ml DataFrame API. Notably, this API requires user and item IDs to be integers, as addressed in Section 3.2. Additionally, the order of the columns was adjusted to match the expected format: user, item, rating (Figure 3.10).

```
1 # Change columns to correct place (user_id, track_id, playcount)
2 test_listening_history = test_listening_history.withColumn("track_id_temp",
3   test_listening_history.track_id).withColumn("user_id_temp",
4     test_listening_history.user_id)
5 test_listening_history = test_listening_history.withColumn("track_id",
6   test_listening_history.user_id_temp).withColumn("user_id",
7     test_listening_history.track_id_temp)
8
9 train_listening_history = train_listening_history.withColumn("
10   track_id_temp", train_listening_history.track_id).withColumn("
11   user_id_temp", train_listening_history.user_id)
12 train_listening_history = train_listening_history.withColumn("track_id",
13   train_listening_history.user_id_temp).withColumn("user_id",
14   train_listening_history.track_id_temp)
15
16 # key = old column, value = new column
17 mapping = {
18     "track_id": COL_USER,
19     "user_id": COL_TRACK,
20     "playcount": COL_COUNT
21 }
22
23 test_listening_history = test_listening_history.select(*[F.col(old).alias
24   (new) for old, new in mapping.items()])
25 train_listening_history = train_listening_history.select(*[F.col(old).
26   alias(new) for old, new in mapping.items()])
27 )
```

Figure 3.10: Column change in data for ALS.

Following the data preparation, some hyperparameters for the ALS model, rank, maxIter, regParam, alpha, and implicitPrefs, were tuned based on the best practices outlined by Long et al [47], however, for the nonnegative we conducted an evaluation experiment in Appendix B, Table B.1. With these values, we were able to train our model with fit, and then predict with transform after removing all of the songs the users had already played (Figure 3.11).


```

1 als = ALS(userCol= COL_USER, itemCol= COL_TRACK, ratingCol=COL_COUNT,
    rank = 10, maxIter = 40, regParam = 0.05, alpha = 60.0,
    coldStartStrategy="drop", nonnegative = False, implicitPrefs = True)
2
3 # Training
4 with Timer() as train_time:
5     model = als.fit(train)
6
7 # Prediction
8 with Timer() as test_time:
9
10     # Get the cross join of all user-item pairs and score them.
11     users = train.select(COL_USER).distinct()
12     items = train.select(COL_TRACK).distinct()
13     user_item = users.crossJoin(items)
14     dfs_pred = model.transform(user_item)
15
16     # Remove seen items.
17     dfs_pred_exclude_train = dfs_pred.alias("pred").join(
18         train.alias("train"),
19         (dfs_pred[COL_USER] == train[COL_USER]) & (dfs_pred[COL_TRACK] ==
20         train[COL_TRACK]),
21         how='outer'
22     )
23
24     top_all_als = dfs_pred_exclude_train.filter(dfs_pred_exclude_train[f"
25     train.{COL_COUNT}"].isNull()) \
26     .select('pred.' + COL_USER, 'pred.' + COL_TRACK, 'pred.' + "
27     prediction")
28
29     # In Spark, transformations are lazy evaluation
30     # Use an action to force execute and measure the test time
31     top_all_als.cache().count()

```

Figure 3.11: Training and Prediction in ALS.

3.5 Evaluation

All the models were evaluated using the same metrics. This was done to compare the models on the same basis. The metrics cover both the quality of the recommendation, their ranking, and the variety of the recommendations.

Gunawardana et al. [51] categorize prediction accuracy into three distinct classes: accuracy of ratings predictions, usage predictions, and rankings of items.

Ratings accuracy is typically assessed using metrics such as RMSE (Root Mean Squared Error) and MAE (Mean Absolute Error) [51]. However, these metrics are not well-suited for our project because our dataset consists of user play counts rather than explicit ratings. Our objective is to predict whether a user will listen to a song, rather than assigning a numerical rating to it. Therefore, usage prediction metrics, such as Precision, Recall, and Hit Rate, are more appropriate for our context. Furthermore, since we generate a fixed number of recommendations for each user, we need metrics that evaluate the relevance of the @K recommended items [51]. While doing the tests we started with 10 outputs which yielded bad results. Therefore we decided to increase the amount of recommendations

which significantly improved the output. This resulted in 3 top-k's that were 10, 25, and 50. For usage prediction, we selected the following four metrics:

Precision@K:

Precision@K is a simple predictive quality metric that evaluates a list of recommendations based on how many relevant predictions (true positives) are in the top K of the recommendation list. The metric is described as [52]:

$$P@K = \frac{\text{Number of Relevant Items in Top } K}{K}$$

Here, the metric takes the number of relevant predictions in the top K and divides it by K to get the percentage of relevant predictions in the top K .

Recall@K:

Recall@K is another predictive quality metric that tells us how many of the relevant predictions for a user are in the top K of the recommendation list. The metric is described as [52]:

$$R@k = \frac{\text{Number of Relevant Items in Top } K}{\text{Total Number of Relevant Items}}$$

Here, the formula takes the number of relevant predictions in the top K and divides it by the total number of relevant predictions for a user to find the percentage of them present in the top K .

MAP:

Mean Average Precision (MAP) is a ranking quality metric that computes the average precision for all ranks in the top K . It is described as [52]:

$$\text{MAP} = \frac{1}{U} \sum_{u=1}^U \left(\frac{1}{|\text{Relevant Items}_u|} \sum_{k=1}^{|\text{Recommended Items}_u|} \text{Precision}@k \times \text{rel}_u(k) \right)$$

Where U is the total number of user. Relevant Items_u is the number of relevant items for user u , $\text{Recommended Items}_u$ is the recommended items for user u , $\text{rel}_u(k)$ is a binary representation of whether item K is relevant (1 if relevant, 0 if not).

Hit@K:

In our case, users are not expected to enjoy every recommended track. There are scenarios, such as when users are looking to discover new songs, where they might feel satisfied even if the recommendation list contains just one track they like. As such, we can use hit rate at K to measure this.

Hit@K gives a binary evaluation of the top K recommendations. It is either 1 or 0 depending on whether any relevant predictions are present in the top K (1 if relevant predictions are found, 0 if not) [51]. It can be described as:

$$HR@K = \min \left(1, \sum_{i=1}^K \text{relevance}(i) \right)$$

NDCG@K:

While usage prediction metrics measure the proportion of relevant items within the top K recommendations, they do not account for the order in which these items appear. Using ranking metrics, allows us to assess the likelihood for a user to engage with items based on their positions in the top K [51]. For this we chose Normalized Discounted Cumulative Gain (NDCG@K) at K , implemented by PySpark [53]. It evaluates a recommendation based on how well it can capture the ranking of relevant predictions in the top K . It is described as [52]:

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

where Discounted Cumulative Gain at K (DCG@K) scores the ranking by giving a logarithmic weight to relevant items lower in the top k . DCG@K is described as:

$$DCG_p = \sum_{i=1}^p \frac{2^{rel_i} - 1}{\log(i + 1)}$$

where rel_i represents the relevance of the i -th item in the recommendation list. Relevance is often binary (1 for relevant and 0 for non-relevant items), as it is in our case, but it can also be based on other criteria, such as raw scores or exponential scaling.

IDCG@K is the ideal DCG@K score, meaning the maximum score the prediction can get. A perfect score for NDCG@K would be one, if not perfect it will be less than one.

In content-based ranking, the ranking system is not inherently available, as it typically relies on content similarity. To address this, the play counts of items were normalized using the following formula:

$$x' = \frac{x}{\sum_{i=1}^N x_i}$$

This normalization process adjusts the play counts proportionally, ensuring consistency across different scales and users. It was applied individually to each user's rating list to transform the raw counts into a comparable ranking. The resulting normalized values were then used to compute ranking scores for each item.

Diversity:

Beyond accuracy, we incorporated a diversity metric to compute how varied the recommended items are for each user. For the collaborative recommendations we used the Recommenders [43] function *diversity* that can be called from two different subpackages from the github. The function can be called from *python_evaluation* [54] or *spark_evaluation* [55], which, as you can guess by the name, are for tests on models that use *pandas* [40] dataframes, such as Cornac's [44], and for models that use *Spark* [45] dataframes, respectively. For content-based recommendations, we developed a custom implementation to accommodate the specific structure of our model and ensure computational efficiency.

Our evaluation encompasses two types of recommender systems: content-based and collaborative filtering. While the methods for scoring each system differ slightly, the core structure of the diversity metric remains consistent across both.

Diversity Metric Definition

Before discussing the differences, let us establish the common foundation of the diversity metric. It is defined as:

$$\text{diversity} = \frac{\sum \text{user_diversity}}{\text{number of users}},$$

because the objective is to calculate the average diversity of recommendations across all users [54].

The key difference between the two approaches lies in how *user_diversity* is computed:

- **Collaborative Filtering:**

$$\text{user_diversity} = 1 - \text{avg_il_sim}$$

where *avg_il_sim* represents the average intra-list similarity, as defined by Ziegler et al. [56]:

$$\text{avg_il_sim} = \frac{\sum_{(i_1, i_2) \in \text{pairs}} \text{similarity}(i_1, i_2)}{\text{number of pairs}}$$

Here, i_1 and i_2 are distinct items (in our case, tracks) belonging to the same recommendation list.

Diving into the *similarity* formula, we use co-occurrence similarity as presented by Zhang et al. [57]:

$$\text{similarity}(i_1, i_2) = \frac{\text{count}(i_1, i_2)}{\sqrt{\text{count}(i_1) \cdot \text{count}(i_2)}}$$

where $\text{count}(i_1, i_2)$ symbolizes the number of times a pair of tracks appears across every user listening history and $\sqrt{\text{count}(i_1) \cdot \text{count}(i_2)}$ represents the square root of the product of each track in the pair.

- **Content-Based:**

$$\text{user_diversity} = \text{avg_il_div}$$

where *avg_il_div* is the intra-list diversity (ILD) and is computed as the average cosine distance between pairs of items in the top- k recommendations. This is defined as:

$$\text{ILD@}k = 1 - \frac{1}{|S_k|(|S_k| - 1)} \sum_{i \neq j} \text{sim}(i, j)$$

Here S_k represents the set of item pairs in the top- k recommendations, and $\text{sim}(i, j)$ is the cosine similarity between items i and j .

For content-based recommendations, we use a modified ILD@ k metric, as described in [58]. Our version replaces Euclidean distance with cosine distance for better alignment with the scale of the diversity formulas:

$$\text{distance}_i = \sum_{\substack{j=1 \\ j \neq i}}^n \|\mathbf{v}_i - \mathbf{v}_j\|$$

Chapter 4

Results

This section presents the evaluation results for all implemented algorithms, as well as a random baseline for comparison.

The evaluation metrics include all of the one mentioned in 3.5, such as Precision@K, Recall@K, Hit@K, NDCG@K, MAP, and Diversity. These metrics were used to assess both the quality and variety of the recommendations produced by the algorithms. Table 4.1 summarizes the performance of the models across the different metrics.

	Random	NMF	BPR	ALS	AutoE	NonE
Precision@10	0.00058	0.00050	0.08643	0.07487	0.01152	0.01196
Precision@25	0.00064	0.00045	0.06475	0.05988	0.00905	0.00943
Precision@50	0.00065	0.00049	0.05010	0.04808	0.00768	0.00805
Recall@10	0.00037	0.00033	0.05548	0.05085	0.00821	0.00851
Recall@25	0.00102	0.00074	0.10368	0.10087	0.01595	0.01658
Recall@50	0.00207	0.00160	0.16060	0.16058	0.02596	0.02708
Hit@10	0.00584	0.00193	0.44442	0.39781	0.09120	0.09334
Hit@25	0.01593	0.00626	0.62534	0.59861	0.16163	0.16655
Hit@50	0.03223	0.01412	0.76218	0.75356	0.24186	0.25257
NDCG@10	0.00061	0.00047	0.09655	0.07943	0.00871	0.00923
NDCG@25	0.00086	0.00061	0.10350	0.09309	0.01175	0.01233
NDCG@50	0.00136	0.00102	0.12979	0.12123	0.01547	0.01623
MAP@10	0.00011	0.00008	0.02923	0.03683	0.00347	0.00368
MAP@25	0.00015	0.00011	0.03856	0.03462	0.00422	0.00447
MAP@50	0.00018	0.00013	0.04510	0.04065	0.00473	0.00501
Diversity@10	0.99842	0.99904	0.81222	0.84395	0.08359	0.09503
Diversity@25	0.99841	0.99898	0.85122	0.87813	0.10384	0.11721
Diversity@50	0.99841	0.99797	0.87967	0.90375	0.12326	0.13847

Table 4.1: Summary of the evaluation of all implemented algorithms.

4.1 Precision@K and Recall@K

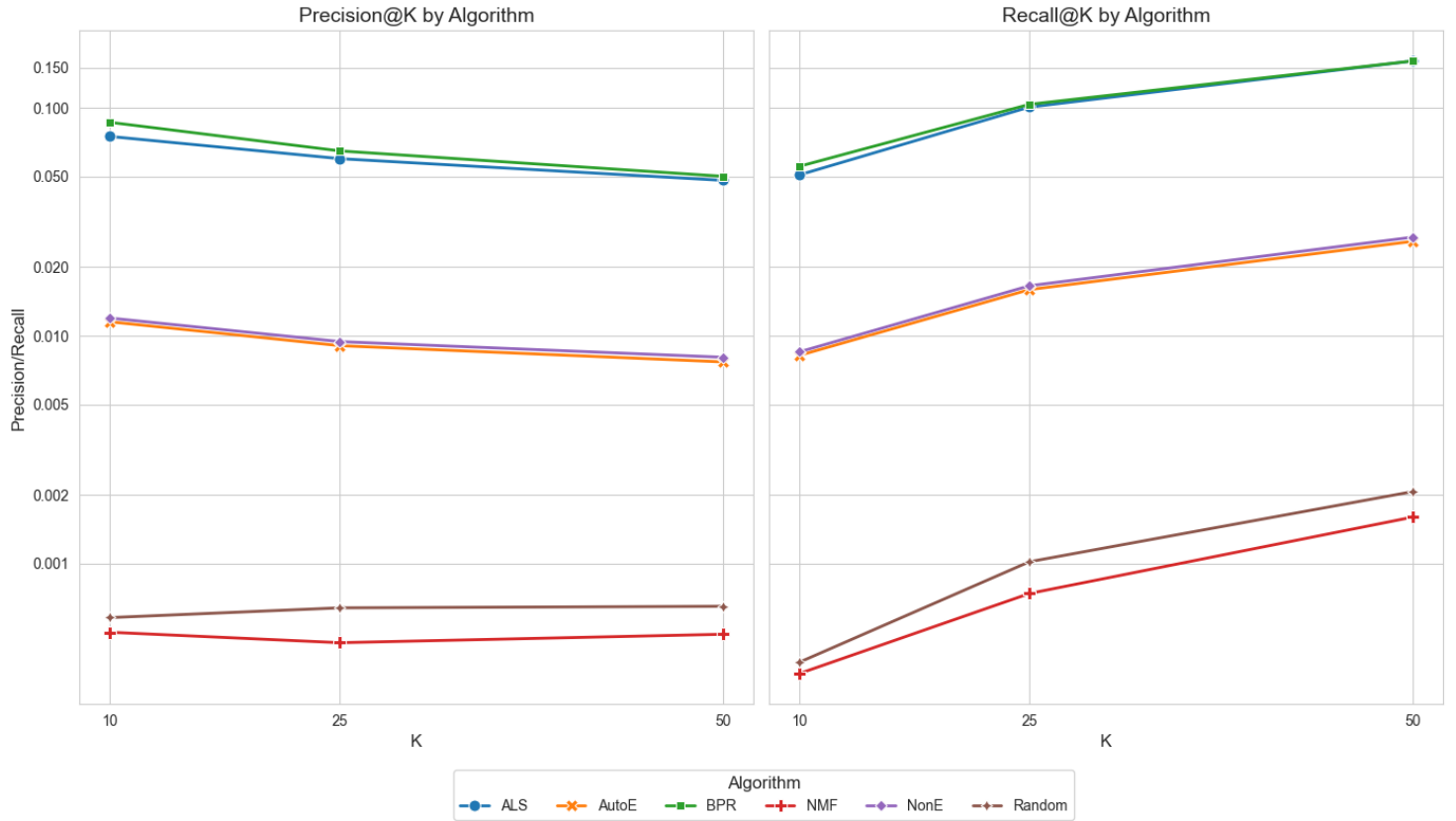


Figure 4.1: Precision@K and Recall@K results for all models with different K values.

The Precision@K and Recall@K metrics provide insights into the relevance of the recommendations, as mentioned in Section 3.5.

As shown in Table 4.1 and Figure 4.1, BPR and ALS algorithms achieve the best overall results for both metrics. For instance, at $K = 10$, BPR achieves a Precision@K of 0.08643 and a Recall@K of 0.05548, while ALS follows closely with a Precision@K of 0.07487 and a Recall@K of 0.05085. This highlights the ability of both models to rank relevant items effectively and retrieve a significant portion of them.

As K increases Recall@K improves for all models, reflecting the inclusion of more relevant items in longer recommendation lists. Conversely, Precision@K decreases slightly due to the larger denominator, which dilutes relevance. Notably, BPR maintains the highest Recall@K at $K = 50$ with a score of 0.16060, slightly outperforming ALS at 0.16058.

The Random baseline performs predictably poorly across all values of K , demonstrating the advantage of model-based approaches. In contrast, NMF exhibits limited effectiveness, achieving the lowest Precision@K and Recall@K among the recommender system models. AutoE and NonE algorithms deliver intermediate results, showing some capability in capturing relevance but not matching the performance of BPR and ALS.

Overall, BPR and ALS stand out for their robustness, particularly in generating high-quality recommendations in both short and long recommendation lists. Their performance

stability across Precision@K and Recall@K emphasizes their suitability for this task.

4.2 Hit@K

Figure 4.2 shows Hit@K indicating if algorithm is successful in including at least one relevant item in K recommendations.

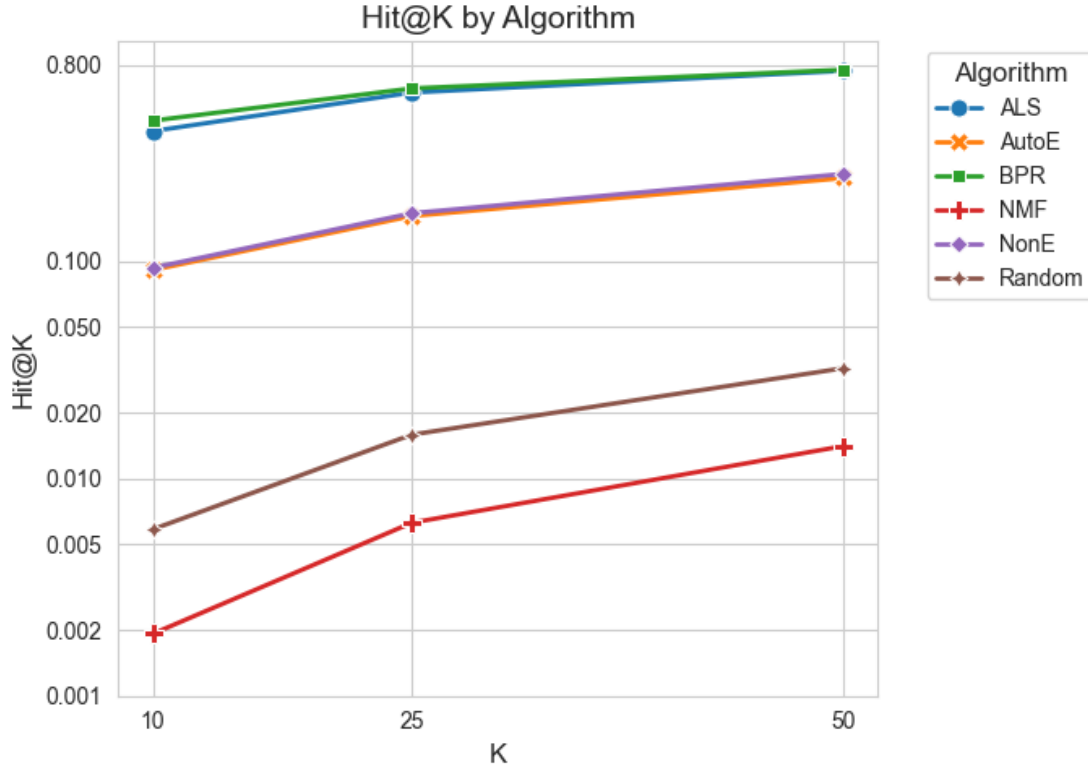


Figure 4.2: Hit@K results for all models with different K values.

Among the collaborative filtering algorithms, BPR and ALS demonstrate the best performance. At $K = 10$, BPR achieves the highest Hit@K of 0.44442, indicating that around 44.44% of users have at least one relevant item in their top 10 recommendations. ALS follows with a Hit@K of 0.39781, showing slightly lower but comparable performance. As K increases to 50, both algorithms improve significantly, with BPR achieving 0.76218 and ALS 0.75356. This trend reflects the natural tendency of Hit@K to increase as more items are included in the recommendation list, thus increasing the likelihood of including relevant items.

NMF on the other hand struggles to provide good results. Even at $K = 50$, NMF achieves a Hit@K of only 0.01412, indicating its inability to include relevant items in recommendations.

Content-based algorithms, while performing better than NMF, still fall significantly behind the BPR and ALS methods. At $K = 10$, AutoE and NonE only achieve Hit@K scores of 0.09120 and 0.09334, and 0.24186 and 0.25257 at $K = 50$.

Overall, BPR and ALS demonstrate best ability to include relevant items for a majority of

users across varying K values, and consistently outperforming content-based algorithms AutoE and NonE.

4.3 NDCG@K

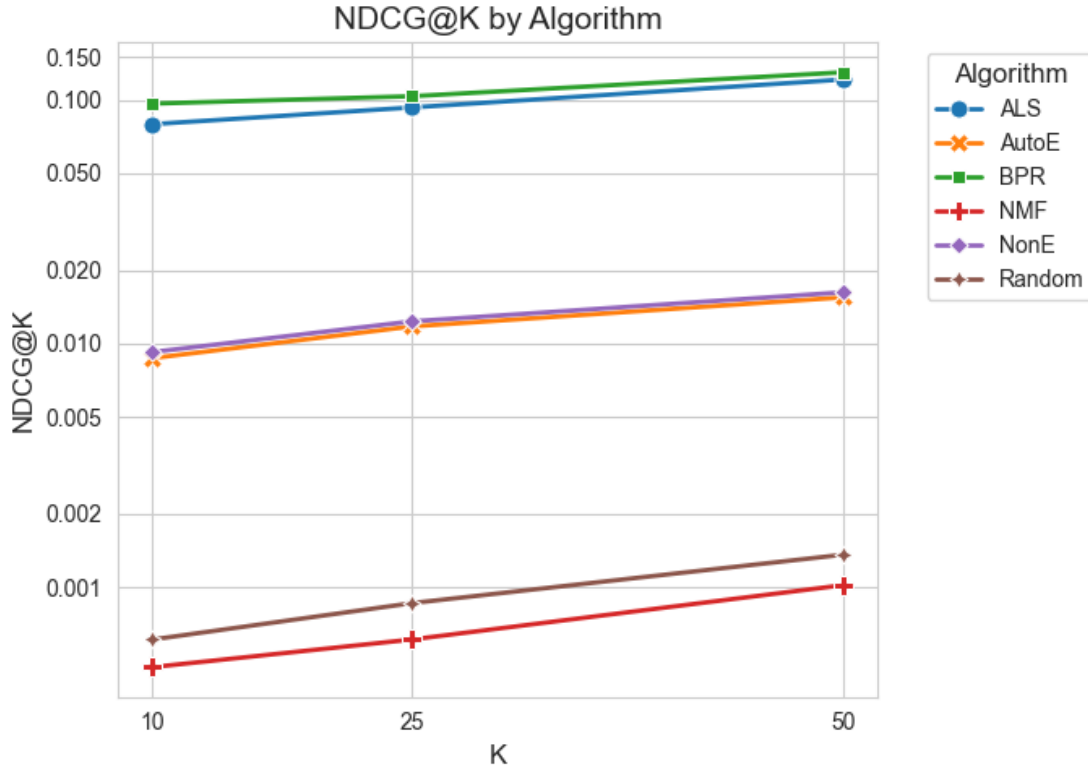


Figure 4.3: Score results of all models for NDCG@K with different K values

In Figure 4.3, a higher NDCG@K indicates that the algorithm places relevant items at higher ranks on average. We observe that NDCG@K increases with larger K values since there is a greater likelihood of capturing relevant items within a broader recommendation set. Among the collaborative filtering algorithms, BPR achieves the highest NDCG@K scores across all K values, indicating better ranking quality. ALS performs slightly worse than BPR. For instance, at $K = 10$, BPR achieves an NDCG@K of 0.09655, while ALS scores 0.07943, and the gap is even smaller at $K = 50$, where BPR achieves 0.12979 compared to 0.12123 for ALS.

In contrast, NMF shows negligible NDCG@K scores, with a value of approximately 0.00102 at $K = 50$. This highlights NMF's inability to rank relevant items effectively and is not much better than random guessing at placing relevant items at the top.

Overall, BPR and ALS show strong ranking performance across all K values and produce good results in ranking relevant items, while NMF fails to rank relevant items effectively.

As seen on figure 4.3 the AutoE and NonE produce similar NDCG scores but due to the way NDCG@K is calculated the scores for it will differ between the content-based models and the collaborative filtering as described in section 3.5. Making it difficult to compare

the score with the other algorithms.

4.4 MAP

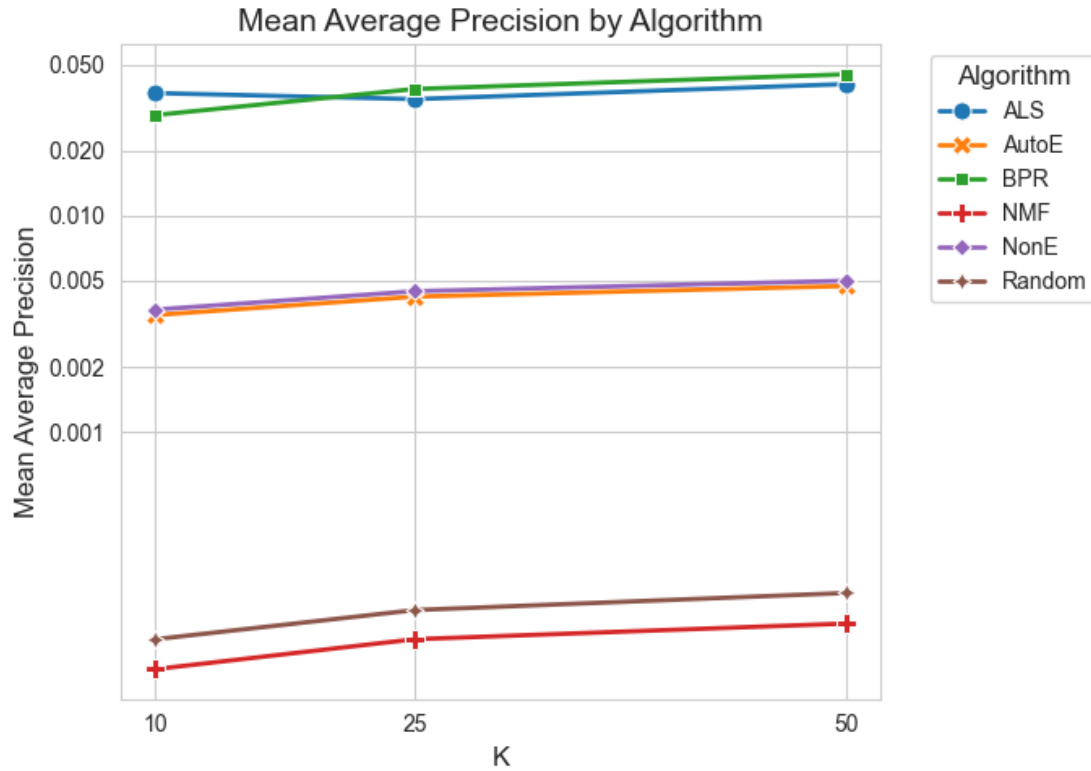


Figure 4.4: Score results of all models for MAP@K with different K values

For the Mean average precision (MAP) a higher score will indicate the algorithm will be better at placing relevant predictions at a higher place in the top K, making it suitable for precision oriented ranking for a user. The data shows that the MAP score slightly increases with the top K with only the ALS having a slight decrease in the score from $K = 10$ to $K = 25$ but then increasing above both scores at $K = 50$. From the graph it can be seen that the BPR and ALS outperform the other four algorithm by a significant margin with both having similar result. Following the BPR and ALS are the AutoE and NonE but still with significantly lower scores than the BPR and ALS algorithms. The Data also shows the NMF algorithm actually performs worse then the random baseline recommendation when it comes to ranking the relevant items in the top K.

To summarize the best performing algorithms are BPR and ALS having scores in the range of 0.02923 to 0.04510 depending on the top K. This means that the best performing algorithms will predict relevant items in the top K for around three to five percent of the users.

4.5 Diversity

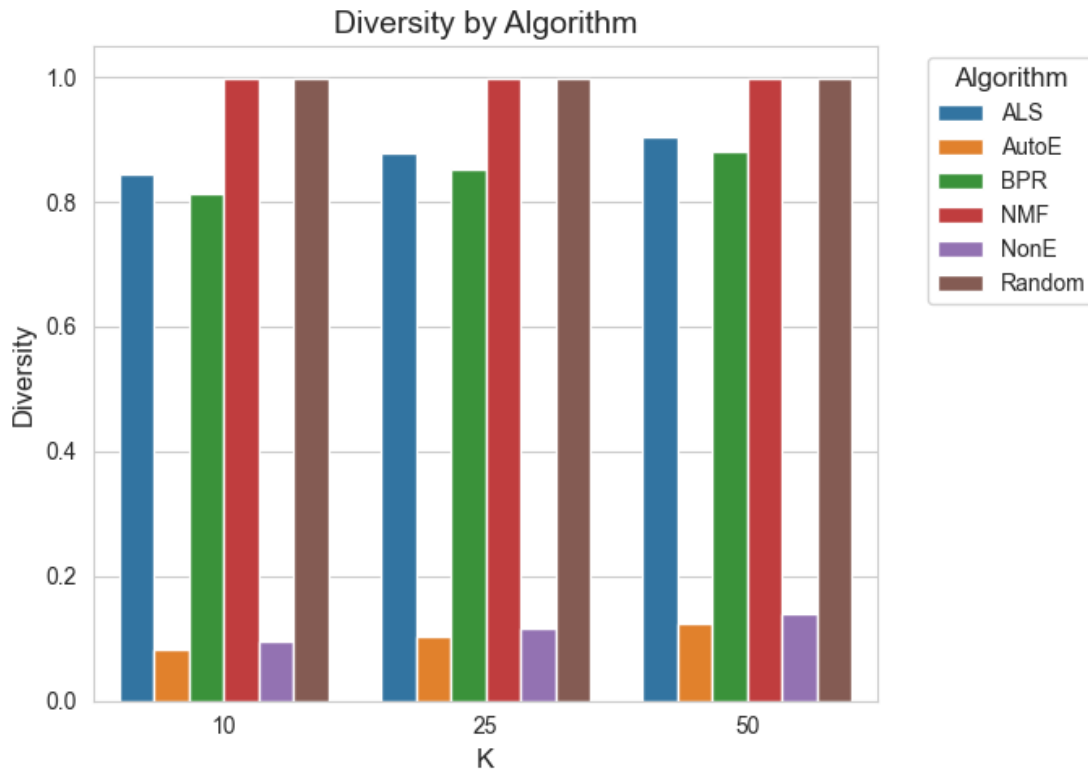


Figure 4.5: Score results of all models for Diversity with different K values

The Diversity graph in Figure 4.5 highlights notable differences between models regarding their ability to recommend diverse items. Random and NMF models achieve the maximum diversity score of 1.0 consistently across all K values (10, 25, 50), indicating that they offer the broadest range of recommendations. For Random, this result is expected as the algorithm generates purely random suggestions. NMF excels in diversity mainly due to the very low precision presented, as seen in the Precision@ K graph (Figure 4.1).

On the other hand, ALS and BPR deliver a high level of diversity, with scores ranging from approximately 0.82 to 0.9. ALS slightly outperforms BPR in this aspect, maintaining its position as a solid compromise between diversity and precision. Both collaborative filtering methods provide recommendations that are diverse but remain focused on user preferences, as reflected by their higher precision compared to other algorithms.

In contrast, content-based models such as AutoE and NonE perform poorly in diversity, with scores remaining between 0.08 and 0.14. These low diversity values reveal that content-based approaches tend to suggest items that are highly similar to each other, which can lead to overly homogeneous playlists, commonly referred to as the *filter bubble* effect [30].

Finally, the diversity scores increase as K increases. This makes sense because by augmenting the playlist size and adding more songs there is a higher probability of the similarity between the songs decreasing and so improving diversity.

Chapter 5

Discussion

This section examines the findings of our study, focusing on the balance between diversity and predictive quality in recommender systems, as well as practical considerations regarding scalability and security. We discuss the trade-offs observed in the results, particularly the interplay between quality and diversity. Additionally, we address challenges faced during dataset preparation and explore security concerns, such as privacy and system vulnerabilities, that impact real-world usability.

5.1 Usability

Our primary goal was to explore how different algorithms influence the diversity of recommendations provided to users. Diversity is a subjective and multifaceted concept. Inner diversity reflects subtle variations within familiar content, while outer diversity involves exposure to entirely new or unexpected items. These dimensions vary significantly between users. In our results, while not explicitly categorized, we understand that inner diversity aligns with moderate diversity scores and higher predictive quality, whereas outer diversity could correspond to higher diversity scores but lower predictive quality.

For example, the Random baseline achieved extremely high diversity scores (close to 0.998), indicating highly varied playlists. However, its predictive quality metrics, such as Precision@K and Recall@K, were negligible. This highlights that pure randomness, while diverse, fails to provide meaningful or engaging recommendations.

In contrast, algorithms like BPR and ALS demonstrated slightly lower diversity (BPR at $K = 50$ with *Diversity* = 0.87967, ALS at $K = 50$ with *Diversity* = 0.90375) while delivering superior relevance. This is consistent with existing literature, suggesting that recommender systems often prioritize precision and quality over diversity, creating less varied playlists but closely aligned with user preferences [59].

The trade-off between diversity and quality was evident across all algorithms. As quality increased, diversity generally decreased. For instance, BPR and ALS achieved strong Precision@K performance at smaller K values but exhibited greater diversity at higher K , where accuracy declined. These varied playlists can offer users opportunities to explore new music and artists, but they risk deviating too far from user expectations. However, lower precision does not always equate to poor user experience, especially when hit rates

remain acceptable.

On the other hand, algorithms such as AutoE and NonE achieved lower diversity while maintaining intermediate accuracy. For example, AutoE recorded a diversity score of 0.12326 at $K = 50$, balancing variety and relevance to some extent. However, its performance fell short of expectations, as it lacked the precision typically associated with the pattern of lower diversity and higher precision, and vice versa. This is due to how our model works. Instead of using the whole user listening history as input for the AutoE, our model works with the input of a single vector of features, meaning that, for it to generate a playlist, we need to calculate the mean of the features across all user-listened songs. Considering this, our model tends to perform better for users with a less diverse taste, since the mean of the features across all user-listened songs is more accurate to the real features of the songs they listen to, and performs worse for users with a more varied listening history. For example, suppose a user listens to some songs with higher values of energy but also others with lower. In that case, the mean of the energy features for those two songs is going to be an average value, while the actual user taste is both songs with higher and lower energy.

For real-world usability, BPR and ALS are well-suited for users seeking personalized recommendations that balance quality and variety. These models avoid excessive repetition while maintaining relevance. For users inclined toward a mix of outer and inner diversity, i.e., exploring music outside their comfort zones whilst maintaining some similarities, collaborative filtering algorithms like BPR and ALS are promising due to the high diversity and relevant precision. By leveraging shared listening histories across users, these algorithms broaden user preferences and enhance discovery.

Our findings highlight the inherent trade-off between diversity and relevance in recommender systems. While highly precise algorithms deliver accurate and personalized playlists, they may constrain users' exploratory potential. Conversely, highly diverse approaches lack the focus needed for meaningful engagement. Achieving a balance between these objectives is crucial for effective recommender systems. Algorithms like BPR and ALS, which perform well on both dimensions, offer a solid foundation for this balance. For platforms prioritizing user exploration and discovery, these algorithms can be further optimized to enhance both novelty and satisfaction.

5.2 Scalability

Within the space of scalability, our project does contain some issues that would have to be solved. Due to resource limitations, we had to restrict ourselves to using pre-made datasets with our recommender algorithms. Although it reduced the time of the project significantly, using pre-made datasets presented us with some other challenges. One of these challenges was that we had to spend quite some time transforming and tailoring the datasets to the algorithms we would be using. The transformation of the datasets happened gradually as we discovered more things within them that would have to be changed to adjust them to the algorithms. One such thing was, for example, the fact that the *Music Info* dataset, which had to have the tags converted into binary values and therefore ended up containing 131 columns in total, which could lead to an increase in the needed computing power. This meant that our development of the algorithms was constantly slowed down by having to wait for transformed versions of the datasets to be

created or due to more computing power being needed. Even though we did not have the resources at our disposal to create our own datasets we could have looked into using ones that were more directly made for use with the algorithms we were using. These factors could potentially avoid reliance on outside sources like UCloud to run our models.

Moreover, the *User Listening History* dataset contained implicit feedback in the form of *playcount*. This meant that we would not be able to conclude upon the user's preferences as precisely as would be the case with explicit feedback and early in the project it led to concerns with how we would be using the dataset. We did, however, find a way of using it with the ALS model by treating implicit feedback as a confidence level. Even though we did manage to get results treating the implicit feedback values this way, we could have looked into explicit feedback data's impact on creating diverse recommendations.

5.3 Security

Although security has not been a major focus of this project, it remains an important topic within recommender systems. The works of Himeur et al. [60] found that the research on the topic should be studied more in-depth and shared their take on the current security challenges within the field of recommender systems.

An aspect of security concerns within recommender systems Himeur et al. [60] highlighted was the end-user's privacy. With recommender systems having access to extensive knowledge about their users, the misuse of these data plays a key part in some of the security concerns within the field. For example, Himeur et al. [60] found that researchers and service providers should look out for the trade-off between recommendation quality and users' privacy. While knowing more about a user would lead to a higher quality of recommendations, it could reach a point where information would become more sensitive and breach the privacy of the user. The research also found that who was exposed to the user data was of great concern. While including third parties in handling the data of a recommendation could be done with good intentions in mind, the data should not be shared with third parties hoping to monetize off the data through the use of for example selling the data or using it for targeted advertising.

Another thing Himeur et al. [60] talked about was some of the ways a recommender system could be attacked. They defined an attack on a recommender system as being - *"An external interference by means of either an automated algorithm or by maliciously intruding a noise in the database which in turn can affect the system prediction, hence resulting in an inaccurate, biased or untrustworthy recommendation."* [60]

One of these types of attacks is injecting fake users or item ratings/item features into the dataset thus way influencing the collaborative filtering algorithms and content-based algorithms, respectively. In the collaborative filtering context, the attackers could use fake users to boost a certain item to trick the system into recommending it more often. Meanwhile, the introduction of fake item ratings or item features could help attackers promote an item or reduce the chance that a competing item is recommended.

Chapter 6

Future Work

This chapter discusses considerations for future work aimed at improving and extending the scope of our recommender models.

In terms of evaluation, there are additional ways to improve and analyze our models using non-accuracy metrics that can further help us evaluate performance in terms of diversity. Gunawardana et al. [51] highlight several such metrics. For instance, **novelty** measures the degree to which recommended items are new or unfamiliar to the user, making it particularly relevant for encouraging exploration in our recommendations. **Serendipity** builds on this by evaluating how surprising and pleasantly unexpected the recommendations are, promoting the discovery of tracks that users might not have anticipated but would still enjoy. Lastly, **coverage** measures how effectively the models include less popular or long-tail items, ensuring a broader range of songs is considered in recommendations. This increases the likelihood of exposing users to new artists or genres. Incorporating these metrics into future evaluations would give valuable insights into which algorithms best balance novelty, surprise, and wide-ranging content, and contribute to richer and more engaging song recommendations.

To further improve the recommendation for users it would be appropriate to look further into other models for recommendation. This includes models that are not either collaborative or content-based but instead are a hybrid of both. Using models that are a hybrid of different methods could enable the recommender system to give more diverse and relevant predictions. An approach to this could be to utilize collaborative filtering to create a user profile and then pass it to a content-based model. This would add more variety to the profiles making the content-based model recommend more diverse.

Our current study relies entirely on offline experiments. While offline evaluations are an important step in filtering out best performing algorithms and tuning model hyperparameters, next step should include user studies and online evaluation. User studies would involve deploying the best-performing algorithms in a controlled environment and have small group of participants interact with the system. Especially when it comes to diversity it would be important to avoid biases to measure the impact of diversity on user experience. Then finally it would be necessary to do online evaluations, for example, A/B testing, by integrating the algorithms into a live system and testing with real users. They provide the most accurate comparisons of algorithms in real-world settings [51]. In these experiments, a small percentage of users could be redirected to different recommender systems, and user interactions and trade-offs between these systems could be analyzed.

Concerning the data processing phase discussed earlier, we observed that the content-based dataset expanded significantly, reaching 131 columns per row. This increase in dimensionality led to a substantial rise in memory consumption. For future work, one approach to mitigate this issue could be to analyze the importance of each tag and identify less significant tags that can be safely removed without compromising the model’s performance. Additionally, the dataset lacked sufficient genre information, with only about 12 genres available. Expanding the variety of genres while simultaneously reducing unnecessary tags could enhance data processing efficiency for the content-based model.

For the user listening dataset, one potential optimization is to aggregate similar user listening histories. This could significantly reduce the dataset’s size. However, such aggregation must be done cautiously, as excessive aggregation might distort the data. For example, averaging playcounts could obscure individual user rankings or unique listening patterns. To further improve the collaborative models, incorporating more implicit (e.g., skips, replays) and explicit (e.g., ratings, likes) feedback into each data point could enhance user profile generation and increase the model’s accuracy.

In a realistic scenario, the data would be utilized in a real-time system, necessitating frequent cleaning and efficient management. Even with the current dataset, the models require significant computational power, requiring the use of a high-performance server such as UCloud for processing. To enable a recommender system like ours, an automatic data management system is essential. This could be achieved by using timestamps for each data entry, which would allow for the removal of outdated data through expiration policies. Furthermore, implementing a balancing system could ensure that a diverse range of user types and songs are consistently represented in the dataset. This would optimize the systems performance while maintaining variety and freshness in recommendations.

Chapter 7

Conclusion

This project aimed to analyze the diversity of recommendations generated by various models within the field of recommender systems. To accomplish this we worked with content-based and collaborative filtering models and aimed to create song recommendations for a user. On these recommendations, we applied a variety of evaluation metrics to get a better understanding of which model/models performed the best, having the diversity metric as the main focus. Working with diversity is, however, a difficult subject since it is a broad term and varies from user to user. Therefore, we tried to look into two of the ways diversity can be interpreted, this being inner and outer diversity. Our work revealed that the collaborative filtering models outperformed the content-based models not only in generating diverse song recommendations but also, to our surprise, in terms of predictive quality metrics. While working on our main objective, we encountered some of the limitations of content-based and collaborative filtering. Content-based stayed within the same recommendation space, causing the *filtering bubble* effect, while collaborative filtering heavily relied on a diverse user group, therefore requiring an extensive amount of data. Additionally, our results emphasize the importance of selecting algorithms that align with the nature of the dataset. The NMF model would probably have performed better had we been working with explicit feedback instead of implicit feedback. This could be seen from the fact that we used negative sampling and binary transformation to "convert" the implicit feedback to explicit feedback, which yielded better results for NMF (see Appendix C).

In conclusion, while our study focused on the influence of algorithms on the diversity of music recommendations, further work is needed to bridge the gap between theoretical diversity and practical user satisfaction. Future research should prioritize involving real users, optimizing models for specific data contexts, and exploring different approaches to enhance both inner and outer diversity in recommendations.

Bibliography

- [1] M. Kunaver and T. Porl, "Diversity in recommender systems - a survey," pp. 154–162, 2017. [Online]. Available: <https://doi.org/10.1016/j.knosys.2017.02.009>
- [2] K. Robinson, D. Brown, and M. Schedl, "User insights on diversity in music recommendation lists," pp. 446–453, 2020. [Online]. Available: https://www.researchgate.net/publication/348432339_User_Insights_on_Diversity_in_Music_Recommendation_Lists
- [3] UndefinedNull, "Million song dataset + spotify + last.fm," 2022. [Online]. Available: <https://www.kaggle.com/undefinenull/million-song-dataset-spotify-lastfm/data?select=Music+Info.csv>
- [4] UCloud, "Ucloud - main page," 2024. [Online]. Available: <https://cloud.sdu.dk/app>
- [5] Y. F. F.O. Isinkaye and B. Ojokoh, "Recommendation systems: Principles, methods and evaluation," pp. 261–273, 2015.
- [6] Google for Developers, "Content-based filtering," 2024. [Online]. Available: <https://developers.google.com/machine-learning/recommendation/content-based/basics>
- [7] P. Xia, L. Zhang, and F. Li, "Learning similarity with cosine similarity ensemble," pp. 39–52, 06 2015.
- [8] IBM, "What is a neural network?" 2024. [Online]. Available: <https://www.ibm.com/topics/neural-networks>
- [9] Google, "Machine learning glossary," 2024. [Online]. Available: <https://developers.google.com/machine-learning/glossary>
- [10] GeeksforGeeks, "Artificial neural networks and its applications," 2024. [Online]. Available: <https://www.geeksforgeeks.org/artificial-neural-networks-and-its-applications/>
- [11] Google, "Neural networks," 2024. [Online]. Available: <https://developers.google.com/machine-learning/crash-course/neural-networks>
- [12] —, "Deep neural network models," 2024. [Online]. Available: <https://developers.google.com/machine-learning/recommendation/dnn/softmax>
- [13] Y. Zhang, "A better autoencoder for image: Convolutional autoencoder." [Online]. Available: https://users.cecs.anu.edu.au/~Tom.Gedeon/conf/ABCs2018/paper/ABCs2018_paper_58.pdf

- [14] A. Dertat, “Applied deep learning - part 3: Autoencoders,” 2017. [Online]. Available: <https://towardsdatascience.com/applied-deep-learning-part-3-autoencoders-1c083af4d798>
- [15] Y. Koren, S. Rendle, and R. Bell, “Advances in collaborative filtering,” New York, NY, pp. 91–142, 2022. [Online]. Available: https://doi.org/10.1007/978-1-0716-2197-4_3
- [16] F. Ricci, L. Rokach, and B. Shapira, “Recommender systems: Techniques, applications, and challenges,” New York, NY, pp. 1–35, 2022. [Online]. Available: https://doi.org/10.1007/978-1-0716-2197-4_1
- [17] A. N. Nikolakopoulos, X. Ning, C. Desrosiers, and G. Karypis, “Trust your neighbors: A comprehensive survey of neighborhood-based methods for recommender systems,” New York, NY, pp. 39–89, 2022. [Online]. Available: https://doi.org/10.1007/978-1-0716-2197-4_2
- [18] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” pp. 30–37, 2009.
- [19] Y. Zhang, “An introduction to matrix factorization and factorization machines in recommendation system, and beyond,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.11026>
- [20] F. O. Isinkaye, “Matrix factorization in recommender systems: Algorithms, applications, and peculiar challenges,” pp. 6087–6100, 2021. [Online]. Available: <https://doi.org/10.1080/03772063.2021.1997357>
- [21] J. Bobadilla, J. Dueñas-Lerín, F. Ortega, and A. Gutiérrez, “Comprehensive evaluation of matrix factorization models for collaborative filtering recommender systems,” p. 15, 2024. [Online]. Available: <https://doi.org/10.9781/ijimai.2023.04.008>
- [22] X. Zhang, X. Zhou, L. Chen *et al.*, “Explainable recommendations with nonnegative matrix factorization,” pp. 3927–3955, 2023. [Online]. Available: <https://doi.org/10.1007/s10462-023-10619-9>
- [23] D. D. Lee and H. S. Seung, “Algorithms for non-negative matrix factorization,” Cambridge, MA, USA, pp. 535–541, 2000. [Online]. Available: <https://proceedings.neurips.cc/paper/2000/file/f9d1152547c0bde01830b7e8bd60024c-Paper.pdf>
- [24] M. E. Haque, S. M. Zobaed, M. E. Tozal, and V. V. Raghavan, “Divergence based non-negative matrix factorization for top-n recommendations,” 2019. [Online]. Available: <https://scholarspace.manoa.hawaii.edu/bitstream/10125/60067/0248.pdf>
- [25] B. Wundervald, “Cluster-based quotas for fairness improvements in music recommendation systems,” pp. 25–32, 2021. [Online]. Available: <https://doi.org/10.1007/s13735-020-00203-0>
- [26] G. Takács and D. Tikk, “Alternating least squares for personalized ranking,” New York, NY, USA, p. 8390, 2012. [Online]. Available: <https://doi.org/10.1145/2365952.2365972>
- [27] H. Li, B. He, M. Lublin, and Y. Perez, “Matrix completion via alternating least square(als),” University Lecture, 2015. [Online]. Available: <https://stanford.edu/~rezab/classes/cme323/S15/notes/lec14.pdf>

- [28] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "Bpr: Bayesian personalized ranking from implicit feedback," Arlington, Virginia, USA, p. 452461, 2009.
- [29] A. Milogradskii, O. Lashinin, A. P. M. Ananyeva, and S. Kolesnikov, "Revisiting bpr: A replicability study of a common recommender system baseline," 2024.
- [30] Q. M. Areeb, M. Nadeem, S. S. Sohail, R. Imam, F. Doctor, Y. Himeur, A. Hussain, and A. Amira, "Filter bubbles in recommender systems: Fact or fallacy – a systematic review," 2023. [Online]. Available: <https://arxiv.org/abs/2307.01221>
- [31] Spotify, "Spotify," 2024. [Online]. Available: <https://www.spotify.com/dk-en/free/>
- [32] Spotify - Safety and Privacy Center, "Understanding recommendations on spotify," 2024. [Online]. Available: <https://www.spotify.com/dk-en/safetyandprivacy/understanding-recommendations>
- [33] Spotify for Developers, "Spotify web api - documentation," 2024. [Online]. Available: <https://developer.spotify.com/documentation/web-api>
- [34] A. S. Gillis, "Data splitting," 2024. [Online]. Available: <https://www.techtarget.com/searchenterpriseai/definition/data-splitting>
- [35] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," pp. 263–272, 2008. [Online]. Available: <https://ieeexplore.ieee.org/document/4781121>
- [36] University of Southern Denmark (SDU), "The sdu escience center offers new platform for research data analysis," 2022. [Online]. Available: https://www.sdu.dk/en/om-sdu/fakulteterne/sundhedsvidenskab/nyhedsarkiv/nyt_sund_2022/ucloud
- [37] UCloud, "Products - ucloud," 2024. [Online]. Available: <https://docs.cloud.sdu.dk/guide/resources-products.html>
- [38] "Miniconda software," 2024, version 24.9.2-0. [Online]. Available: <https://docs.anaconda.com/miniconda/>
- [39] "Anaconda software distribution," 2024, version 2024.10-1. [Online]. Available: <https://docs.anaconda.com/anaconda/>
- [40] W. McKinney, "pandas: a foundational python library for data analysis and statistics," 01 2011.
- [41] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," pp. 2825–2830, 2011.
- [42] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: <https://www.tensorflow.org/>

- [43] A. Argyriou, M. González-Fierro, and L. Zhang, "Microsoft recommenders: Best practices for production-ready recommendation systems," New York, NY, USA, p. 5051, 2020. [Online]. Available: <https://doi.org/10.1145/3366424.3382692>
- [44] A. Salah, Q.-T. Truong, and H. W. Lauw, "Cornac: a comparative framework for multimodal recommender systems," Jan. 2020.
- [45] The Apache Software Foundation, "Pyspark," 2024, version 3.5.3. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/index.html>
- [46] Recommenders team, "recommenders predict ranking function," 2023. [Online]. Available: https://github.com/recommenders-team/recommenders/blob/main/recommenders/models/cornac/cornac_utils.py
- [47] J. Long, L. Dirick, N. Solomon, and A. Soto, "Building recommendation engines with pyspark." [Online]. Available: <https://www.datacamp.com/courses/recommendation-engines-in-pyspark>
- [48] Preferred.AI, "Cornac bpr documentation," 2023. [Online]. Available: https://cornac.readthedocs.io/en/stable/api_ref/models.html#bayesian-personalized-ranking-bpr
- [49] —, "Cornac nmf documentation," 2023. [Online]. Available: https://cornac.readthedocs.io/en/stable/api_ref/models.html#module-cornac.models.nmf.recom_nmf
- [50] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar, "Mllib: machine learning in apache spark," p. 12351241, Jan. 2016.
- [51] A. Gunawardana, G. Shani, and S. Yogev, "Evaluating recommender systems," New York, NY, pp. 547–601, 2022. [Online]. Available: https://doi.org/10.1007/978-1-0716-2197-4_15
- [52] A. Jadon and A. Patil, "A comprehensive survey of evaluation techniques for recommendation systems," 2024. [Online]. Available: <https://arxiv.org/html/2312.16015v2>
- [53] The Apache Software Foundation, "Rankingmetrics - pyspark documentation," 2024, accessed: 15-Dec-2024. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.mllib.evaluation.RankingMetrics.html>
- [54] Recommenders team, "recommenders python evaluation," 2024. [Online]. Available: https://github.com/recommenders-team/recommenders/blob/main/recommenders/evaluation/python_evaluation.py
- [55] —, "recommenders spark evaluation," 2023. [Online]. Available: https://github.com/recommenders-team/recommenders/blob/main/recommenders/evaluation/spark_evaluation.py
- [56] C.-N. Ziegler, S. M. McNee, J. A. Konstan, and G. Lausen, "Improving recommendation lists through topic diversification," 2005. [Online]. Available: <https://api.semanticscholar.org/CorpusID:655375>
- [57] Y. C. Zhang, D. Ó. Séaghdha, D. Quercia, and T. Jambor, "Auralist: introducing serendipity into music recommendation," 2012. [Online]. Available: <https://api.semanticscholar.org/CorpusID:2956587>

- [58] D. B. Kyle Robinson and M. Schedl, "User insights on diversity in music recommendation lists," 2020. [Online]. Available: https://www.researchgate.net/publication/348432339_User_Insights_on_Diversity_in_Music_Recommendation_Lists
- [59] A. Anderson, L. Maystre, I. Anderson, R. Mehrotra, and M. Lalmas, "Algorithmic effects on the diversity of consumption on spotify," New York, NY, USA, p. 21552165, 2020. [Online]. Available: <https://doi.org/10.1145/3366423.3380281>
- [60] Y. Himeur, S. S. Sohail, F. Bensaali, A. Amira, and M. Alazab, "Latest trends of security and privacy in recommender systems: A comprehensive review and future perspectives," p. 102746, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404822001419>

Appendix A

BPR Hyperparameter Experiment

Algo	Precision@k	Recall@k	NDCG@k	MAP	Diversity	
BPR_Rec@10	0.08643	0.05548	0.09655	0.02923	0.81222	
BPR_Rec@25	0.06475	0.10368	0.1035	0.03856	0.85122	
BPR_Rec@50	0.0501	0.1606	0.12979	0.0451	0.87967	
BPR_Cornac@10	0.02569	0.01684	0.02768	0.00624	0.85228	
BPR_Cornac@25	0.02024	0.03303	0.03135	0.00787	0.87343	
BPR_Cornac@50	0.01697	0.05499	0.04168	0.0091	0.89371	

Table A.1: Summary of different BPR hyperparameters performance. BPR_Rec represents BPR with the recommenders [43] hyperparameters and BPR_Cornac represents the default values in the Cornac documentation [48].

As you can see in Table A.1, the BPR model with the recommenders [43] hyperparameters excels in every ranking metric in all K values when compared to the BPR model with Cornac [48] default hyperparameters.

Considering this, we chose the model with the best performance in ranking metrics, BPR with recommenders [43] hyperparameters, so we can better relate all models in terms of diversity.

Appendix B

ALS NonNegative Parameter Experiment

Algo	Precision@k	Recall@k	NDCG@k	MAP	Diversity	
ALS_NN@10	0.05903	0.03936	0.06221	0.02799	0.82991	
ALS@10	0.07487	0.05085	0.07943	0.03683	0.84395	

Table B.1: Summary of ALS@10 performance. ALS_NN represents ALS with Implicit Preferences and Non-Negative options set to true and ALS represents ALS with only Implicit Preferences set to true.

As shown in Table B.1, the ALS model without *NonNegative* parameter outperforms the other in all metrics. We only evaluated with K equal to 10 because that is the best value for precision, the metric we wanted to compare.

With this said, our choice was the model with the best results, ALS without *NonNegative* parameter.

Appendix C

NMF Experiment with Binary Transformation and Negative Sampling

Algo	NegS	Precision@k	Recall@k	Hit@k	NDCG@k	MAP	Diversity
NMF@10	10	0.00126	0.00078	0.01244	0.00125	0.00023	0.99592
NMF@10	25	0.00215	0.00131	0.02085	0.00200	0.00034	0.99499
NMF@10	50	0.00377	0.00231	0.03387	0.00355	0.00065	0.99323
NMF@10	100	0.00547	0.00348	0.04518	0.00501	0.00098	0.98768
NMF@10	200	0.00780	0.00515	0.06048	0.00716	0.00153	0.96978
NMF@10	400	0.01541	0.01057	0.10880	0.01441	0.00346	0.95058

Table C.1: NMF@10 performance for varying numbers of negative samples.

Table C.1, the results of the NMF experiment demonstrate the impact of introducing negative sampling for NMF model at various levels (10, 25, 50, 100, 200, and 400 negative samples per user). This improvement indicates that the additional negative samples help the model distinguish relevant from irrelevant items more effectively.

Appendix D

Random Recommender Implementation

```
1 # # Random Recommender
2 # Set the random seed for reproducibility
3 np.random.seed(SEED)
4
5 # Get all unique tracks from the training data
6 all_tracks = set(train[COL_TRACK].unique())
7
8 # Create a dictionary mapping of each user and tracks they've interacted
  with
9 user_seen_tracks = train.groupby(COL_USER)[COL_TRACK].apply(set).to_dict
  ()
10
11 recommendations = []
12
13 for user, seen_tracks in user_seen_tracks.items():
14     # Get the set of unseen tracks for the user
15     unseen_tracks = list(all_tracks - seen_tracks)
16
17     # Assign random prediction scores to the sampled tracks
18     prediction_scores = np.random.rand(len(unseen_tracks))
19
20     # Create recommendation entries with user, track, and prediction
  score
21     user_recommendations = pd.DataFrame({
22         COL_USER: user,
23         COL_TRACK: unseen_tracks,
24         'prediction': prediction_scores
25     })
26
27     # Sort by prediction score in descending order
28     user_recommendations = user_recommendations.sort_values(by='
  prediction', ascending=False)
29
30     # Append to the list of recommendations
31     recommendations.append(user_recommendations)
32
33 # Concatenate all user recommendations into a single DataFrame
34 all_prediction_sorted_random = pd.concat(recommendations, ignore_index=
  True)
```

Figure D.1: Random recommender implementation